SCHOOL OF TECHNOLOGY

FINAL YEAR PROJECT II

CHAPTER 4: IMPLEMENTATION PLAN

TechPulse

Evidence-led implementation schedule and operational plan.

Unit Code & Title: STU 4201 - Final Year Project II
Program: Bachelor of Science in Data Science
Student Name: Tony Kenga
Registration Number: 24/03652
Supervisor: Dr. Rufus Gireka
ORCID: 0009-0007-6899-8590
Date Submitted: 29th September 2026
Faculty: School of Technology (SoT)

Contents

4.1 Implementation Schedule 3

4.1.1 Work packages and baseline . . . 4

4.1.2 Milestone gates and schedule control . . . 5

4.1.3 Development-mode run evidence . . . 5

4.1.4 Schedule acceptance . . . 6

4.2 Installation & Conversion Plans 7

4.2.1 Installation sequence . . . 7

4.2.2 Conversion inventory . . . 8

4.3 Software and Hardware Installation Plans 9

4.3.1 Software baseline . . . 10

4.3.2 Proposed hardware profile . . . 11

4.3.3 Proposed capacity review and acceptance . . . 12

4.4 Activities of Conversion 13

4.4.1 Conversion activities . . . 14

4.4.2 Completion criteria . . . 15

4.5 System Conversion Strategy 16

4.5.1 Strategy comparison . . . 17

4.5.2 Rollback matrix . . . 18

4.6 Training Plan 19

4.6.1 Outcomes and delivery . . . 19

4.6.2 Training assessment rubric . . . 20

4.6.3 Materials and refresh . . . 20

4.7 Training Methods 21

4.7.1 Method sequence . . . 21

4.7.2 Feedback and accessibility . . . 22

4.7.3 Effectiveness . . . 22

4.8 Software Maintenance Plan 23

4.8.1 Maintenance categories . . . 24

4.8.2 Release, backup and monitoring . . . 25

4.8.3 Maintenance acceptance . . . 25

4.9 Change Management Plan 26

4.9.1 Lifecycle . . . 26

4.9.2 Change class and review . . . 27

4.9.3 Request-to-close procedure . . . 27

4.9.4 Verification by affected contract . . . 28

4.9.5 List of figures and tables . . . 78

4.1 Implementation Schedule

The proposed baseline is seven one-week periods, P1-P7, or seven weeks of elapsed work. The supervisor may adjust dates and durations through change control. Person-hours below are planning estimates, not measured effort. The proposed owners are student developer, supervisor and operational user.

Evidence rule: repository behavior and the development-mode run below are observations. Periods, owners, hours, gates, installation, training and operating controls are proposals. The local manifest marks `research_grade`=false and identifies deterministic proxy tables. Survey usage shares are not direct sentiment; generated adoption and company mappings are not evidence of real enterprise adoption.

The period labels are relative checkpoints, not calendar promises. P1 confirms scope and establishes runtime; P2 completes source inventory and contract review; P3 closes feature and target decisions; P4 completes initial candidate training; P5 reviews evaluation and explanation output; P6 integrates and exercises the dashboard; P7 completes pilot acceptance and handover. Preparation may overlap, but the proposed critical dependency chain cannot be bypassed without recording a change to the evidence plan.

Effort estimates include review preparation and evidence capture, not only coding. They are rounded planning hours because the repository has no time log. Before adopting them, compare each package with academic deadlines, research-data access and user availability. Any adjustment should state whether scope is reduced, duration extended or workload reassigned.

At each weekly check-in, record owner, planned period, evidence status, blocker, next action and gate effect. If a milestone cannot proceed, keep downstream work provisional. Do not report a model or dashboard as accepted merely because later tasks have started. An approved scope change should update the schedule, training material and handover assumptions together.

Use five proposed task states: not started, in progress, blocked, ready for review and accepted. Only accepted work advances a gate. A blocked task names the dependency, owner and next decision date; ready for review identifies the reviewer and evidence location. These labels are a simple reporting convention proposed for this project, not an existing repository feature.

Package A produces the scope and environment record. Package B records the source mode, provenance, schema and quality limits. Package C captures predictor meaning, target rules and leakage checks. Package D records candidate training artifacts and split metadata. Package E retains metric eligibility and explanation review. Package F validates the user workflow against the artifact bundle. Package G records conversion and restore evidence. Package H assembles the runbook and unresolved issues. This ties effort to reviewable deliverables rather than code volume.

Package boundaries reduce the chance that a late dashboard task hides an unresolved data question. For example, model training can be explored during development, but its output remains provisional until the source and target decisions pass M2 and M3. If a task must be repeated after a source change, record the repeat effort against the same package and update its period estimate.

4.1.1 Work packages and baseline

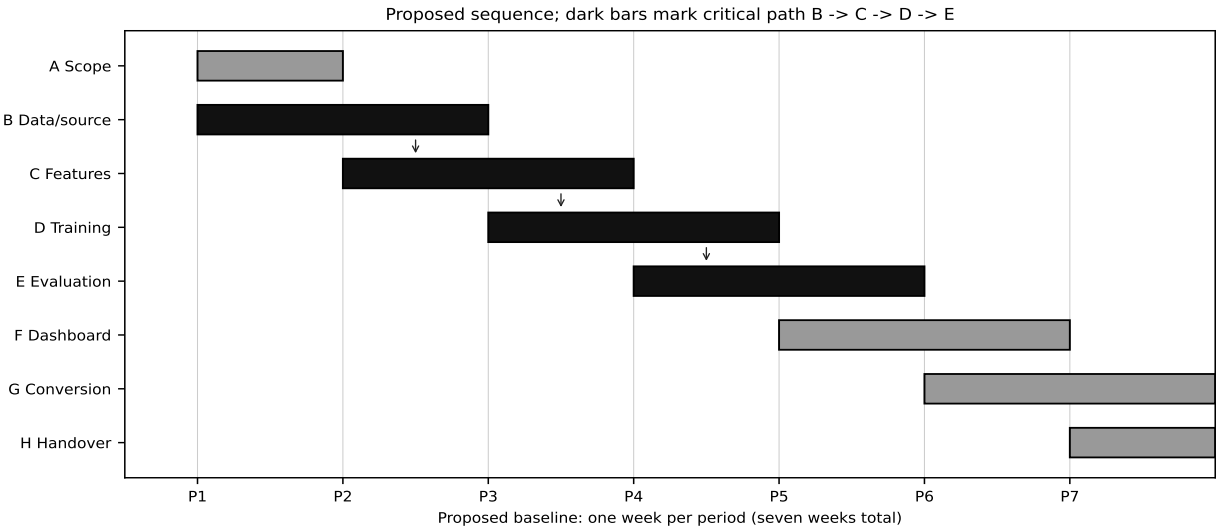
A practical task breakdown for package B starts with an input inventory, then a column-level schema map, then profiling of keys, dates and missingness. Package C turns the approved mapping into a reviewed feature and target definition. Package E checks whether the evaluation slice can support the metrics being shown. Package F traces at least one output through the dashboard to its originating run. These substeps are proposals for review, not evidence that they have been performed.

Table 4.1: Proposed work packages, effort and ownership

Package	Work and completion evidence	Periods	Hours (estimate)	Proposed owner
A Scope	Confirm purpose, source mode and environment; approved scope note.	P1	10	Student developer
B Data	Profile provenance, schema, coverage and proxy boundaries; data note.	P1-P2	18	Student developer
C Features	Review seven predictors, target thresholds and leakage; decision record.	P2-P3	20	Developer; supervisor review
D Training	Run supported models; retain split, preprocessing and artifacts.	P3-P4	18	Student developer
E Evaluation	Review baselines, eligible metrics, warnings and explanations.	P4-P5	14	Developer; supervisor review
F Dashboard	Check artifact compatibility, views and score wording.	P5-P6	14	Developer; user review
G Conversion	Trial source/bundle conversion, task walkthrough and rollback.	P6-P7	12	Developer; operational user
H Handover	Deliver runbook, versioned bundle and issue list.	P7	10	Developer; supervisor

Figure 4.1: Proposed seven-period Gantt schedule

The A-H sequence and P1-P7 baseline appear in Figure 4.1. Packages B-C-D-E form the proposed critical path: source acceptance precedes feature/label review, model training and evaluation.



4.1.2 Milestone gates and schedule control

Owners report accepted evidence rather than subjective percent complete. A task closes only when its evidence exists and the proposed reviewer accepts it. If a dependency slips, record the cause, downstream effect and revised baseline before moving the gate. Estimates are workload aids, not delivery guarantees.

Table 4.2: Milestone gates M1-M7 and minimum proceed evidence

Gate	Proceed evidence	Hold condition	Decision owner
M1 Environment	Repeatable runtime; paths and secrets understood.	Dependency conflict or unsafe credential handling.	Student developer
M2 Data	Source mode, lineage, permission and proxy status reviewed.	Unknown lineage, schema mismatch or proxy presented as observed.	Supervisor
M3 Features /labels	Definitions, time windows and leakage reviewed.	Future fields enter predictors or fallback is unexplained.	Supervisor
M4 Training	Split, class support, preprocessing and artifacts retained.	Unsupported split or irreproducible preprocessing.	Student developer
M5 Evaluation	Baselines, eligible metrics, warnings and limits reviewed.	Metric invalid or not traceable.	Supervisor
M6 Integration	Version-matched model, predictions, explanations and dashboard.	Stale bundle or misleading score wording.	Developer
M7 Handover	Guide, maintainer, rollback and open issues delivered.	No maintainer or critical issue open.	Supervisor

The evidence gates are summarized in Table 4.2. Schedule risks are managed as triggers, not assumed away.

4.1.3 Development-mode run evidence

The canonical entry point `pipeline/run_pipeline.py` was executed with `DATABASE_URL` unset, selecting local `CSV` mode. `PASS` means the step returned without raising. It does not establish research validity or acceptable model performance.

```
SHAP input capped at 500 training samples for runtime control.

TechPulse Pipeline Summary
Step | Status
--- | ---
```

```
FR-01 data_ingestion.load_and_validate_datasets | PASS
FR-02 eda.generate_eda_report | PASS
FR-03 feature_engineering.compute_feature_matrix | PASS
FR-04 labelling.assign_trajectory_labels | PASS
FR-05-FR-09 model_training.train_all_models | PASS
FR-10 evaluation.compare_and_select_best_model | PASS
FR-11 shap_analysis.run_shap_analysis | PASS
```

Observed artifacts include `outputs/data_sources.json`, `data_provenance.json`, `data_quality_report.json`, `eda_report.json`, `feature_schema.json`, `target_leakage_columns.json`, `labelling_summary.json`, `model_comparison.csv`, `model_comparison.md`, `evaluation_summary.json`, `best_model_selection.json`, `global_feature_importance.csv`, `technology_predictions.csv`, `shap_per_prediction.json`, `shap_values.npy`, `shap_bar_chart.png`, `shap_summary_beeswarm.png` and the four model confusion-matrix plots. The models directory contains `best_model.joblib`, candidate model files and the `SHAP` explainer; logs contains the timestamped pipeline log and stage logs. The evaluation artifact warns that the chronological test set lacks Declining cases, so multiclass `ROC-AUC` is not statistically meaningful. No performance score is claimed.

4.1.4 Schedule acceptance

The supervisor reviews data, target, evaluation and academic claims; the operational user reviews task usability; the developer retains manifests, logs, environment and output identifiers. These are proposed assignments, not evidence of a formal board or completed project work.

In PostgreSQL mode, check connectivity and required tables with a read-only account. Keep `DATABASE_URL` out of notebooks, terminal screenshots and reports. If the source is unavailable, do not silently switch a research run to local fallback; verify the manifest after ingestion and stop if its mode differs from the approved plan. Store only redacted configuration with the handover.

A second person should reproduce installation from the run card. The reviewer checks dependency list, command, source mode and output directory, then opens the dashboard from the same run bundle. Successful installation establishes an engineering environment, not data rights, label validity or predictive performance.

A schedule review also checks whether each deliverable has an observable acceptance artifact and a reviewer who can make the decision in time. If an artifact depends on data access, note the access request as a predecessor rather than leaving it as an informal assumption. The student can prepare a fallback engineering task, but cannot substitute proxy data for research evidence without approval.

4.2 Installation & Conversion Plans

Installation and conversion are staged and reversible. TechPulse supports local `CSV` development and PostgreSQL through `DATABASE_URL`. A configured database is not automatically research-grade: source meaning, access, schema and provenance still require review.

Start from the repository runtime evidence instead of upgrading packages opportunistically. Use the configured CI or Docker environment as the first reproducibility reference, retain the package resolution, and record any deviation. If installation fails, classify the failure as interpreter, dependency, native library, path, permission or network related; preserve the error before selecting a remedy. A broad package upgrade can change serialized-model compatibility and should follow maintenance/change review.

After ingestion, inspect the quality report and source manifest before launching the dashboard. Confirm that dates parse as expected, required keys are available, reported missingness has been reviewed, and proxy fields remain identified. If one table is unavailable or mapped ambiguously, stop at the conversion gate; do not produce a seemingly complete dashboard by filling unknown values without an approved rule.

4.2.1 Installation sequence

Keep source mode explicit at every entry point. A developer should state whether `DATABASE_URL` is available before running; the pipeline also writes a data-source manifest, which is the runtime evidence to inspect. If a local fallback is expected, verify the documented `CSV` inputs and their intended use. If a database path is expected, stop rather than silently using fallback data when credentials or tables are missing.

Record the working directory because several repository scripts resolve paths relative to the project root. Run documented commands from the intended root, check that generated files appear in outputs/ rather than replacing data/, and identify any stale artifacts left from earlier runs. A stale prediction file can make a dashboard appear populated after a new pipeline attempt fails.

Keep logs and console output with the run package, but redact private connection strings and tokens. An error record should include the step name, exception type, message and timestamp where available. When fixing setup, change one cause at a time and rerun the failed step or canonical pipeline; avoid unrelated version changes that obscure the original failure.

Table 4.3: Installation sequence, checks and acceptance evidence

Step	Action	Check / retained evidence	Owner
Prepare	Use clean checkout; select mode; record Python version.	Files, runtime and mode recorded.	Developer
Configure	Create isolated environment and install requirements; keep secrets out of source.	Resolved packages; secret review.	Developer
Select source	Use `CSV` for development; approve access/schema before DB mode.	Manifest records mode; credentials private.	Developer; supervisor
Validate	Run pipeline; inspect provenance, quality report and logs.	Summary, warnings and logical tables reviewed.	Developer
Launch	Open dashboard only from coherent generated bundle.	Views and unavailable states checked.	Developer; user
Accept	Review limitations and restoration location before use.	Decision and prior bundle restorable.	Supervisor

The PostgreSQL loader expects `so_questions`, `dev_sentiment`, `fortune500_stacks`, `company_profiles`, `tech_metadata` and `question_company_mapping`. A legacy schema diagram does not replace checking the live loader contract. Use a redacted configuration example. The supplied Dockerfile uses Python 3.11; badges and package minimums do not prove compatibility on every operating system.

4.2.2 Conversion inventory

Conversion evidence is consolidated in Section 4.4 to avoid a second checklist for the same mapping, validation and promotion work. The conversion owner inventories the source, confirms permissions and maps each field to the current loader contract before transforming any data. Keep the pre-conversion files read-only and record schema, date coverage, keys, missingness and any proxy derivation in the manifest.

Before rebuilding features and labels, review whether date parsing, technology normalization, aggregation windows or missing-value handling change the meaning of an observation. After the run, reconcile stable keys and periods from source to output, inspect class support and confirm target-derived columns remain outside predictors. Record exceptions with an owner and reviewer; unresolved meaning or lineage issues defer promotion.

For model and dashboard conversion, preserve one coherent run bundle with its preprocessing, model, predictions, evaluation and explanation artifacts. Rehearse the dashboard task and restore the prior bundle before approving cutover. Section 4.4 gives the release evidence sequence; the installation steps above remain a separate runtime acceptance.

Installation checks are summarized in Table 4.3; conversion acceptance follows Section 4.4. Promotion requires a reviewed source decision, successful rerun, quality review, coherent artifacts and a rehearsed restore. Exit status alone cannot accept converted data. No completed research migration is claimed.

Resolve packages in an isolated environment and retain the resolved list with the run identifier. Minimum pins in `requirements.txt` are lower bounds, not a fully locked environment. Where CI, Docker and a workstation differ, document the discrepancy rather than assuming parity.

Check the qualitative workstation recommendation against actual workload. Record peak memory and disk use during ingestion, features, training and explanations. Keep free space for both accepted and candidate bundles so rollback does not require overwriting files. Any numeric operating threshold should come from observed runs and be approved for the intended environment.

For shared deployment, separately review network exposure, account permissions, retention and backup ownership. A dashboard and container recipe do not establish secure hosting. A local Streamlit launch or Docker build is not evidence of an access-control design. Capture deployment as a separate gate before exposing real users or research data.

A dependency record should make future recreation possible without copying environment secrets. Record the Python executable and resolved packages, identify whether the run used the Docker image, and list optional services that were not used. If a package must be upgraded to resolve a defect, preserve the prior environment record and state which stages were rerun.

Before handing a workstation to another maintainer, confirm that paths do not depend on a personal downloads folder, credentials are not embedded in scripts, and generated artifacts are not mistaken for checked-in source. Any shared host should have an access owner, recovery contact and a decision about retention before real user data are placed there.

The installation record should include enough context to reproduce both success and failure: project revision if available, command, working directory, selected source mode, environment and output location. If Git metadata is absent, say so and identify the local source snapshot used. Avoid copying full logs into the report when they contain sensitive paths or credentials; retain a redacted evidence reference instead.

4.3 Software and Hardware Installation Plans

Software evidence is specific; target hardware limits are not. This plan does not invent a minimum CPU, memory, disk or network threshold.

The software record should include interpreter version, operating system, resolved Python packages, container base where used, database client connectivity and the command that generated the bundle. The repository minimum constraints are not an exact lock file. If separate environments are used for the dashboard, pipeline and document generator, record each one and avoid implying that a single successful build verifies all three.

Before adding an operating-system package or changing a model library, identify which pipeline stage needs it and whether existing configuration already supplies it. Re-run the appropriate tests

and canonical pipeline after a runtime change, then check model loading and dashboard rendering from the newly generated artifacts. Keep the previous environment description until the new release is accepted.

On a workstation shared by project members, separate personal workspace files from source-controlled inputs and generated artifacts. Use a named project directory with predictable relative paths, and avoid copying secrets into data folders that may be archived. The project has no evidenced enterprise secret manager; if one is introduced, document its owner and retrieval procedure without placing credentials in this plan.

If later deployment becomes necessary, perform a distinct sizing and access review using observed workload and expected user concurrency. Do not extrapolate requirements from one local run. Test startup, shutdown, storage persistence, log rotation and restore in the intended host before calling the environment production-ready.

4.3.1 Software baseline

Distinguish tools required for the TechPulse application from tools required to reproduce the report. The pipeline requires its Python dependencies; PostgreSQL is optional for the local development path; Docker is optional unless container delivery is chosen. The document generator separately requires ReportLab, matplotlib and pypdf, and uses the bundled Roboto Mono font for technical identifiers. Keep this distinction in the setup guide so that users do not install unused services.

For package installation, record the command, interpreter path and result. If an optional dependency is absent, determine whether the selected execution mode needs it before adding packages. For example, database drivers are relevant to PostgreSQL ingestion, while local `CSV` mode should not require credentials. A reproducible setup should say which optional components are intentionally excluded.

Where the repository's tested runtime differs from the user workstation, reproduce in the configured container or CI environment first. If the container cannot access a required file or native dependency, record the issue and assess whether the container definition or application path should change. Do not present the image build alone as end-to-end runtime proof.

A future hosted environment should have separate acceptance for network boundary, identity, logs, persistence and backup. These are architecture decisions beyond the current local runner. The operational owner should approve the environment and document rollback before public or institutional access is enabled.

Table 4.4: Software requirements and evidence source

Component	Repository evidence	Proposed installation/check
Python	Dockerfile uses 3.11; README badge says 3.10+; CI is compatibility evidence.	Use CI/container baseline; record version and resolved packages.
Packages	requirements.txt includes data, ML, SHAP and Streamlit libraries.	Isolated environment; retain package resolution; rerun after changes.
Database	Optional PostgreSQL uses DATABASE_URL and six expected tables.	Approved account; verify schema and read permission.
Dashboard	Streamlit reads generated model/output artifacts.	Launch after run; check views and unavailable states.
Container/CI	Dockerfile and GitHub Actions workflows are present.	Build at repository root if Docker is available.
PDF generator	ReportLab, matplotlib, pypdf and bundled Roboto Mono.	Run docs/implementation_plan/build.py.

The software baseline in Table 4.4 separates repository evidence from the proposed operator check.

4.3.2 Proposed hardware profile

A concrete sizing exercise begins by identifying the deployment task: local engineering, supervised research analysis or shared dashboard use. Record the current device and operating system, available processor and memory, free storage, filesystem location and network path to the selected source. These are observations to gather, not minimum values defined by this plan. Note whether the run uses local CSVs or queries PostgreSQL, since their resource and network needs differ.

Run one representative development workload in the intended environment and record elapsed time and peak resource use for ingestion, feature generation, training, evaluation and SHAP. Note the input size and selected model alongside each observation so later measurements are comparable. The existing SHAP cap of 500 training samples is a runtime control in the code; record its effect on the run rather than interpreting it as a capacity guarantee.

Check storage headroom for raw inputs, intermediate matrices, logs, model files, plots and both current and candidate bundles. Confirm that a failed run will not overwrite the last accepted outputs. If the filesystem is network-mounted, verify that permissions, file-lock behavior and expected read/write access work from the pipeline account. Retain only approved data in local caches and temporary folders.

For a shared host, estimate concurrent users and refresh frequency with the operational owner before selecting hardware. Measure dashboard response and resource use under that intended pattern; a single-user development run does not answer this question. If the target environment differs materially from the development device, repeat the setup and pipeline checks there. Recommend a profile only after measurement, and label it as specific to the tested workload.

Use a supported 64-bit workstation with enough memory and free disk for inputs, intermediate matrices, models, dashboard outputs and the Python environment. This is a qualitative recommendation, not a tested minimum. Record device, OS, available memory/disk and

largest-run duration; refine the profile from measured use. The SHAP sample cap is a runtime choice, not a machine requirement.

Use least-privilege data access and keep credentials separate from shared documents. The repository does not evidence production hosting, automated backup, model registry or deployed access control; each requires an owner and verification before operational use.

4.3.3 Proposed capacity review and acceptance

Record the sizing observation as a small environment note: device and operating system, Python/container environment, input mode and size, stage completion, elapsed time, peak memory if available, peak temporary storage if available, and whether the output bundle was complete. Repeat the observation when the data volume, model set or hosting pattern changes materially. State the measurement tool and any limits of the observation; do not extrapolate to larger concurrent workloads.

The proposed reviewer considers both capacity and recovery. The selected environment should complete the agreed workload without losing logs or overwriting the accepted bundle, and the operator should be able to locate the generated files. If capacity is insufficient, first identify the bottleneck and decide whether the workload, host or optional explanation step should change. Changes that affect study outputs require research review, not only an infrastructure adjustment.

Installation acceptance therefore records a decision such as suitable for local development, suitable for supervised analysis, or not yet suitable for shared use. These decision labels are recommendations to distinguish use contexts, not measured ratings already assigned to TechPulse hardware. Reassess before any deployment or data-volume increase and preserve the evidence that supports the recommendation.

Accept installation when runtime and packages are recorded, source mode is explicit, the pipeline produces coherent artifacts, and a reviewer can launch the dashboard from that set. Retain console output, logs and redacted configuration. Proxy fallback must remain labelled development mode.

Preserve an immutable input copy and a manifest of origin, extraction time, intended use and schema. Record row counts and date coverage before transformation, then compare them with the quality report and feature matrix. Explain material differences from filtering, normalization or parsing instead of forcing totals to match.

Check joins and identity normalization with representative aliases, including differences in punctuation and case. Determine whether missing observations mean zero activity or unknown activity. Those interpretations affect features and labels, so mapping decisions that change meaning require research review.

For each conversion exception, record affected field, likely impact, disposition and reviewer. Keep any waiver narrow and time-limited. If rerunning changes artifacts, preserve both

manifests and identify the source, code or environment difference before promoting the new result.

Review the conversion manifest before interpreting any metrics. It should make visible whether input tables were observed, derived or generated, and whether data are complete for the claimed period. Keep research and development bundles in separate locations or label them so strongly that a user cannot confuse the two. This is particularly important when both bundles contain files with identical names.

Investigate parsing and join behavior with small traceable samples as well as aggregate reports. A small sample can reveal timezone assumptions, date coercion, key collision or an unexpected fallback. Record the checked sample identifiers and expected transformation in the conversion note, subject to the source data's access rules.

For each transformation step, identify whether it is deterministic and which input changes should cause a rebuild. Record the file or module responsible, the expected output and the validation that detects drift. This reduces the risk that a partially refreshed directory mixes an old evaluation report with new predictions. If the pipeline cannot rebuild an artifact from its documented inputs, treat reproducibility as an open maintenance issue.

4.4 Activities of Conversion

Conversion moves from development checkout toward approved data, repeatable model bundle and accepted workflow. Inspect and map inputs first, transform second, validate outputs, rehearse with users, then decide on promotion. Preserve source and manifest for traceability.

Use a conversion manifest that records source identity, source mode, extraction time, intended period, permitted use, schema version and any proxy derivation. Store the manifest beside the generated run outputs. For a database source, also retain a safe description of the query/table contract and account scope; do not copy connection secrets. A source change should create a new run identity rather than overwrite the old manifest.

Validate data in stages. First check file/table presence and parseability. Next review required keys, duplicate identities, null patterns and date ranges. Then inspect normalization and aggregation by technology and observation month. Finally compare labels, feature order, train/test periods and prediction outputs. A failure at an early stage should stop later interpretation, even if downstream code can technically continue.

Keep a short exception log with a stable issue identifier, affected input, likely consequence, proposed handling, reviewer and closure decision. Distinguish an accepted limitation from a corrected defect. If a source field is unavailable, do not backfill it with a proxy unless the derivation is explicit and the resulting source mode remains accurately described.

Conversion review should include examples from the beginning, middle and end of the period, not only aggregate totals. For selected technology-month records, trace source rows through

normalized identity, feature values, label eligibility and dashboard presentation. This spot check can reveal unit or join mistakes that aggregate counts conceal. Record chosen examples with enough identifiers to reproduce them while protecting restricted data.

4.4.1 Conversion activities

Data profiling should use the repository quality report as a starting point, not the only review. Examine column types, key uniqueness, non-null rules, date parsing and category coverage. Compare expected and actual logical tables for database mode. For `CSV` mode, record the exact source files and whether optional survey data were present. A missing optional file should not be silently interpreted as a measured zero.

Preserve transformation lineage from raw fields to output features. For each engineered predictor, document source column(s), aggregation unit, temporal window, missing-value treatment and whether normalization uses information from outside the training period. If the implementation fits transformations globally, that may affect validation interpretation and should be reviewed before making a research claim.

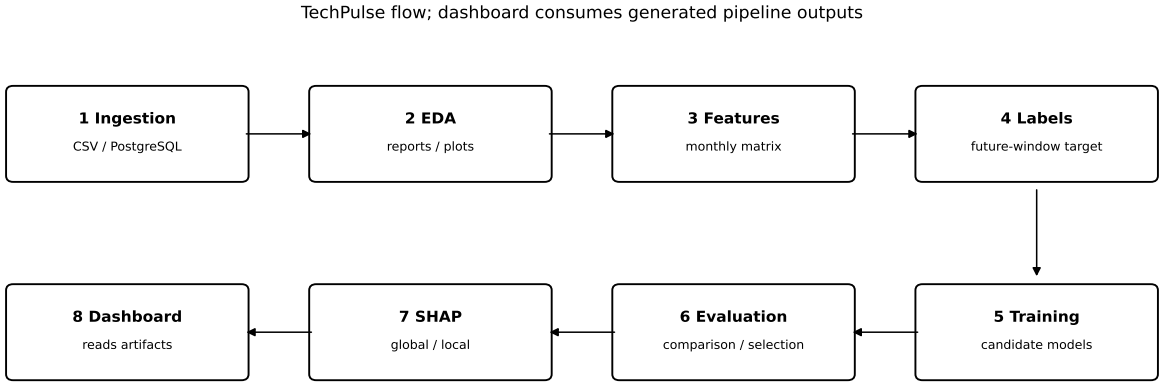
Label conversion deserves its own review because the labeler may use future-window fields or legacy labels. Inspect representative boundary cases around configured growth and decline thresholds, plus missing-window cases. Confirm target columns are excluded from predictors and that a chronological split does not allow future observations to inform earlier training. Record class support by split.

Reconciliation should compare stable keys and date windows rather than only file sizes. Investigate unexpected duplicate technology-month keys, rows excluded by date parsing, normalized aliases that collide, and outputs with absent classes. Any unresolved discrepancy is a conversion exception with an owner and disposition, not a footnote hidden in a run log.

Table 4.5: Conversion activities and release evidence

Activity	Procedure	Acceptance evidence
Discover	Inventory source, owner, permission, periods, definitions and proxies.	Manifest with owner, extraction date, schema and use limits.
Map	Map fields to current ingestion contract; document types, keys, dates and null rules.	Reviewed mapping and sample rows.
Transform	Use existing ingestion, feature and labelling code; do not edit generated predictions by hand.	Run ID, configuration and artifacts.
Validate	Inspect quality, date ranges, feature order, labels, split and leakage exclusion.	Exceptions resolved or accepted; review record.
Reconcile	Compare source/transformed counts and selected records; explain dropped/duplicate rows.	Reconciliation note.
Rehearse	Load coherent bundle; test normal and unavailable dashboard states.	User task record and issue list.
Promote/revert	Supervisor accepts use; retain prior bundle and restore on failed check.	Decision and rollback evidence.

Figure 4.2: TechPulse runtime pipeline and dashboard flow



The eight stages shown in Figure 4.2 reflect `pipeline/run_pipeline.py` plus the dashboard consuming generated artifacts.

`src/common.py` defines seven predictor columns and separate target-leakage columns. Confirm feature creation/training preserves this separation after migration. The labeler uses future/recent monthly activity thresholds and has a legacy fallback when future fields are absent. Establish which path ran and whether its labels match the approved research definition.

Local data include generated adoption-stack, metadata and question-company mapping proxies. Developer-survey rows represent usage shares, not direct sentiment. Preserve those distinctions in each manifest and approve rights/definitions before source changes.

4.4.2 Completion criteria

The conversion evidence in Table 4.5 is accepted only after exceptions are resolved or bounded, agreed user tasks succeed, and restoration is rehearsed. If mapping or label rules remain uncertain, retain development status and defer promotion. Record decision owner and run identifier.

Choose strategy by cost of interruption and uncertainty in the incoming source. A demonstration with disposable proxy data can use a rehearsed direct switch, while a research source warrants staged acceptance and comparison. Do not run both sources through one dashboard directory unless each artifact set is isolated and its source/run identity is visible.

Before cutover, agree who may stop the release, what constitutes a critical defect and where the prior bundle is stored. Verify that the restore copy contains mutually compatible code/configuration, model and outputs. A partial restore can be more misleading than a visible failure because the dashboard may combine new predictions with old explanations.

After promotion, monitor the first user tasks and inspect the newly written manifest. Revert if the source mode is wrong, outputs are incomplete, a key interpretation changes without approval or the agreed task cannot be completed safely. Record the observed trigger and recovery result; only then reopen the change for correction.

The conversion approver chooses the strategy after considering expected outage, data sensitivity, comparison validity, user readiness and restore effort. Capture rejected alternatives and why the selected method fits the present project stage. Revisit the choice if project scope changes from classroom demonstration to research operation or shared service.

A successful rollback rehearsal should start from the candidate state, execute the documented restore steps and verify the previous bundle by its manifest and artifact set. Record elapsed steps only if observed. If rollback requires undocumented developer knowledge or manual file substitutions, it is not yet a dependable recovery path.

A conversion decision should identify whether the proposed release is for engineering demonstration, academic evaluation or operational use. These uses have different acceptance criteria. A local proxy run can support engineering demonstrations, while academic conclusions require appropriate source and methodology review; operational use additionally requires access, support, backup and user acceptance decisions.

4.5 System Conversion Strategy

No single cutover method fits every context. Keep local development for engineering checks, qualify approved database data separately, then expose a coherent bundle to a small user group before wider use. Choose based on reversibility, source maturity and impact.

Use direct cutover only when interruption is harmless, the run is disposable and the restore action has been exercised. This may suit a local classroom demonstration, but it is a poor default for a new research source because source/schema errors can invalidate downstream labels and comparisons.

A parallel review should use clearly separated directories and manifests for the old and new run. Compare source mode, date coverage, row counts, feature schema, label distribution and dashboard task behavior. A difference is a review prompt, not proof that one source is correct. Supervisors decide whether a changed estimand or population remains suitable for the academic question.

Phased conversion is appropriate where independent stages can be accepted in sequence. Do not promote the model until source and target decisions pass review, and do not promote the dashboard until it reads the intended bundle. A pilot is useful only when named users can report issues and someone is available to respond; otherwise it is merely a restricted release.

Set an explicit stop rule before starting cutover. Example proposed triggers are wrong source mode, missing required table, incoherent artifact identifiers, a critical interpretation change without approval, or failure of an agreed task. Record the trigger observed, who stopped promotion and whether the old bundle was restored. These are recommended controls, not evidence of an existing production service.

4.5.1 Strategy comparison

During the strategy decision, estimate the consequence of a failed cutover, availability of a parallel environment, amount of duplicate operation and cost of returning to the last accepted bundle. Do not choose a method only because it is fastest. The decision record should name the source mode, intended users, effective date or period, acceptance authority and any conditions that restrict use.

A parallel comparison should align observation periods and identify definitions that changed. If raw counts or predictions differ, investigate provenance, schema, normalization, missingness, target eligibility and model version before interpreting the difference. Numerical agreement is not required for acceptance where the source legitimately changed, but the supervisor must decide whether the comparison remains meaningful.

A pilot should have a limited audience, an issue contact and a defined review point. Ask pilot users to perform specific tasks and report confusing language or missing outputs. Pilot completion is not inferred from merely opening the dashboard. If no operational user is available, record that usability is unassessed and avoid broad operational claims.

Keep rollback materials separate from the new release, label their version clearly and verify access before the cutover window. Confirm the dashboard can be pointed back to the prior bundle without manual editing of predictions. After rollback, check that it loads the matched model, outputs and explanations; document any changed state before users resume.

Table 4.6: Conversion strategy comparison and recommended use

Strategy	Advantage	Risk/cost	Proposed use
Direct cutover	Fast; single active path.	Highest impact if source or artifacts fail.	Only noncritical demonstration after backup/rehearsal.
Parallel run	Compare old/new before promotion.	Extra effort; invalid if data periods/modes differ.	First research-source promotion; compare provenance/workflow.
Phased components	Move source, model and views in stages.	Needs strict version matching.	Recommended: source, features/labels, bundle, then users.
Pilot group	Limits initial user exposure.	Pilot may not represent everyone.	Use after M6 with support and rollback.

Recommended sequence: phased conversion, short parallel review of the first approved research source and a pilot before wider dashboard use. Do not compare predictions without run identifiers because source mode, period and feature contract may differ.

4.5.2 Rollback matrix

Table 4.7: Rollback triggers and recovery

Trigger	Immediate response	Recovery evidence	Decision owner
Lineage/schema not approved	Stop promotion; keep accepted bundle.	Approved manifest/mapping and successful rerun.	Supervisor
Pipeline fails or artifacts incomplete	Do not launch; capture traceback and run ID.	Rerun and coherent artifact manifest.	Developer
Dashboard loads mismatched files	Withdraw new bundle; restore prior coherent version.	Matching IDs and successful walkthrough.	Developer
Unreviewed label/metric change	Suspend claims; communicate version boundary.	Supervisor approval and updated method.	Supervisor
User task fails	Pause rollout; correct instructions/interface.	Task and comprehension reassessed.	Operational user

Rollback preserves failed evidence: retain logs, manifest, rejected bundle identifier and cause. Correct through change control and resume only after review.

Set training dates only after the relevant environment and example bundle are stable. The student developer needs repeated practice with setup, logs, artifact matching and restore; a supervisor needs evidence interpretation and review tasks; an operational user needs navigation, cautious interpretation and issue reporting. Tailor exercises by role rather than asking every learner to complete the same technical procedure.

Provide a brief reference sheet for each release and identify the person who answers questions during the first use. Keep the teaching example clearly marked as development mode and prevent proxy-generated records from being mistaken for observations. A source change should trigger a short provenance refresher even if the screen layout is unchanged.

Training records should use the minimum information needed to assess competence. Record the task and outcome, not unnecessary personal details. If a learner misses a critical limitation, defer that user's access to interpretive outputs until practice and reassessment demonstrate understanding.

Before training, verify each handout against the current interface and artifact names. Provide a clear contact for questions and explain how users can distinguish a current release from an archived example. If the exercise bundle is synthetic or proxy-based, place that fact beside the exercise itself, not only in an introductory lecture.

A short refresher is recommended after source-mode, target, risk-score or dashboard-navigation changes. The trainer should state what prior understanding remains valid and what changed. Keep the user task list stable where possible so that reassessment can focus on changed behavior rather than a redesigned course.

Measure training against the outcomes in the rubric, not an assumed pass rate. The plan contains no historical duration or learner result beyond proposed session estimates. If a session is shortened or a learner does not complete the task, record the exception and schedule follow-up

rather than marking the objective met. Keep sample outputs and answer guidance version-matched to the current release.

4.6 Training Plan

Training is planned for the student developer, supervisor/evaluator and operational user. It is prospective; repository evidence does not establish completed training. Schedule near integration and repeat after material source, score or workflow changes.

Sequence sessions so the developer has a stable, repeatable environment before teaching others. The supervisor session uses the same manifest and evaluation report that the user-facing session will cite. The operational user session should use a safe sample and finish with an independent task, not merely watch the trainer navigate. Schedule a short follow-up after the learner has had an opportunity to use the guide.

Prepare role-specific objectives in advance. Developer objectives emphasize repeatability, file provenance and recovery. Supervisor objectives emphasize evidence boundaries, class support, temporal split and metric eligibility. User objectives emphasize navigation, meaning of displayed values, limitations and issue reporting. The maintainer objective emphasizes access ownership and which artifacts must move together.

Accessibility and practical constraints should shape delivery. Provide digital and printable instructions, readable type, explicit keyboard/mouse steps where relevant, and an alternative to live high-bandwidth sessions. If language or terminology is unfamiliar, use a glossary and worked example without simplifying away the scientific caveat.

Allow the learner to ask questions without penalizing uncertainty. A question about missing Declining cases or proxy adoption should be treated as useful risk detection. If an answer is not established by repository evidence, say that the claim requires supervisor review and capture it as an open issue rather than improvising a definitive explanation.

4.6.1 Outcomes and delivery

Training objectives should be observable and role-specific. A developer should be able to describe the setup and identify a failed stage; an evaluator should be able to explain why a metric is unavailable for a split; an operational user should be able to distinguish trajectory class from confidence and risk wording. These are proposed learning outcomes and should be reviewed with the intended participants before sessions are scheduled.

Prepare a trainer copy and learner copy of the run card. The trainer copy includes expected checkpoints and common errors; the learner copy gives concise actions and an issue route. Use a development-mode bundle for practice, label it visibly, and do not distribute private database configuration. If users need research-mode training, prepare a separately approved example and explain access boundaries.

A short diagnostic at the beginning can reveal prior familiarity with Python, dashboards and model terminology. Use the response to adjust pacing, not to infer competence. Allow additional supported practice for setup or recovery tasks because these are necessary for safe use even when the learner does not interpret model outputs.

Schedule the supervisor's evidence review before the student presents model conclusions. The reviewer should have the manifest, split metadata, class counts, warning output and methodology note in advance. Training can then focus on actual artifact interpretation rather than hypothetical performance numbers or an unsupported claim of validity.

Table 4.8: Training plan, learning outcomes and effort

Audience	Outcome	Delivery/proposed duration	Owner
Student developer	Install, select mode, run pipeline, trace artifacts and restore bundle.	Hands-on setup; 2 x 2-hour sessions.	Developer; supervisor observes
Supervisor/evaluator	Interpret provenance, target, split, class support and warnings.	Evidence workshop 2h; review 1h.	Developer delivers
Operational user	Navigate dashboard; distinguish trajectory, risk and confidence; report issue.	Walkthrough 90m; reassessment 30m.	Developer; user practices
Maintainer	Reproduce environment, preserve bundle, update dependencies and restore backup.	Runbook rehearsal 2h before handover.	Developer; supervisor accepts

4.6.2 Training assessment rubric

Table 4.9: Assessment rubric and pass evidence

Skill	Observed task	Pass evidence	If not demonstrated
Run	Select mode and execute documented command.	Manifest mode and expected artifacts present.	Repeat checklist; resolve dependency issue.
Data meaning	Explain observed versus proxy limits.	Identifies local proxies and bounds claims.	Retrain on provenance; defer research use.
Model meaning	Explain labels, confidence, risk and metric warning.	Does not call confidence calibrated or output validated.	Repeat examples; revise wording.
Dashboard task	Find technology, trend and explanation; report limitation.	Completes task without misleading inference.	Practice and retest after correction.
Recovery	Restore previous accepted bundle.	IDs match runbook; dashboard loads it.	Do not accept handover before rehearsal.

Record learner, date, trainer, outcome and follow-up. Passing shows the stated task only, not ML certification or model validity. No attendance or participant outcome is claimed.

4.6.3 Materials and refresh

Prepare a run card, provenance explanation, label glossary, task steps and support route. Explain the risk-score calculation and that confidence is not established as calibrated probability. Refresh after changes to data mode, labels, score wording, navigation, access or restoration; version materials with the release.

Demonstration should expose decision points, not just keystrokes: why the source mode is chosen, how a manifest is checked, what makes a bundle coherent and when a run must stop. During practice, allow the learner to make a recoverable mistake, read the error and follow the

issue route. Avoid exercises that require real secrets or unapproved research data.

Use a think-aloud dashboard exercise. Ask the learner to locate a technology, describe the displayed class and confidence, open the explanation, and state one limitation. The trainer notes where the learner hesitates or infers more than the artifact supports. Feedback then updates the run card or interface, and the same task is repeated to confirm the correction.

Provide written instructions in plain language and a compact checklist for time-pressured operation. Where practical, offer the same task without requiring a high-bandwidth video call. Keep a version number on each material so users can identify instructions that no longer match a released workflow.

Facilitators should resist answering unsupported questions with plausible-sounding conclusions. Use the manifest, source files and code as evidence; when they do not settle a research definition, record the question for supervisor decision. This models the same evidence discipline expected when learners interpret dashboard output.

Keep practice mistakes recoverable: use a disposable run directory or copied bundle, and reset it between learners. Do not ask a learner to modify the accepted model or source data. After practice, compare the observed steps with the run card and update instructions where a reasonable learner would take a different path.

Use the same defined terminology in the `CLI` summary, manifest, dashboard and training material. If the interface uses a shortened label, the glossary should map it to the underlying class or quantity. This consistency helps learners recognize that a displayed number is a model output with known limits, not an independently verified real-world fact.

4.7 Training Methods

Use demonstration followed by hands-on practice. A lecture introduces pipeline stages and terms, but competence requires performing tasks in the intended environment. Separate software operation from research interpretation.

4.7.1 Method sequence

Show environment, configuration boundary and source manifest. Run the canonical pipeline while explaining its stages. Ask the learner to repeat it and locate manifest, evaluation summary and prediction file. Trace a dashboard value to its artifact; discuss a proxy or missing-class scenario; rehearse defect reporting and rollback.

Practice with development data. Keep research credentials out of handouts and screenshots. Provide source manifest and known warnings with the dashboard rather than relying on verbal caveats.

A practice exercise should have a defined starting state, task, expected evidence and reset path. Keep an instructor copy with the expected result and a separate learner copy without the answer.

Exercises should use repository-generated outputs or clearly labelled examples; do not fabricate a metric or prediction to make the task easier.

Teach recovery as part of normal operation. If a learner encounters a missing file or failed stage, have them stop, retain the message, check source mode and use the issue route. Do not encourage repeated reruns that overwrite evidence or hide the first failure. Explain which errors are safe to retry and which require maintainer review.

Use structured observation notes: task attempted, step reached, assistance provided, result and follow-up. A group discussion can surface confusing words, but individual task observation is needed before claiming a person can perform a role-critical task. Keep feedback focused on the instruction or interface as well as the learner's familiarity.

After an interface update, repeat a small affected task with a user. After a target, feature, source or score change, retrain on interpretation and comparability. This proportionate approach keeps minor layout corrections from triggering a full course while preserving safeguards for changes that alter meaning.

Include one assessment task in which the right answer is to stop and escalate. For example, a manifest that identifies local development proxies should not be used to make enterprise-adoption claims. The learner should be able to explain the evidence boundary and find the approval route. This tests responsible judgment as well as navigation.

If a user repeatedly misunderstands a term, review the interface label and surrounding explanation before assuming more training will solve it. Collect the exact phrase that caused confusion, revise wording with the supervisor and operational user, and test the revised screen in the same task context.

4.7.2 Feedback and accessibility

Ask learners what the system can and cannot conclude. Correct confusion between risk and probability of decline, SHAP contribution and causation, or proxy and observed adoption. Provide readable written steps and a low-bandwidth alternative where feasible.

Collect task completion, error type, questions and corrective action. If time-to-completion is measured, label it as an observation. These are future records, not invented results. The user and supervisor review revised instructions where misunderstanding affects interpretation.

4.7.3 Effectiveness

Retain a method when the role completes the task and describes its limits. Otherwise simplify steps, add a worked example or change the interface, then reassess. Attendance alone is not competence. Limit training-record access to the project team.

A proposed cadence is to review run failures and artifact completeness after every pipeline execution, review dependency and access changes before each release, and review backup restoration at a supervisor-approved interval. These are recommendations to adopt; no such

monitoring cadence is claimed as already operating. The owner should record exceptions and unresolved issues rather than silently editing a generated artifact.

Separate maintenance severity from research impact. A user-interface defect may block a task without changing a model definition; a seemingly small label-threshold edit may invalidate comparison with prior results. Escalate semantic changes even if code diff size is small, and preserve experiment identifiers when retraining.

When a maintainer becomes unavailable, hand over the environment description, artifact locations, access owner and recovery steps before removing access. Test the handover with the receiving maintainer. Avoid shared personal credentials; ownership of repositories, database accounts and backups should be explicit.

For each release, keep an inventory of model, preprocessing, prediction, evaluation and SHAP files that are expected to travel together. Check that output timestamps and identifiers are coherent before publication. If only part of the pipeline is rerun, label the resulting artifacts partial and prevent them from replacing the accepted complete bundle.

A proposed maintenance review can also track whether a library update changes warnings, serialization or dashboard behavior. When a change has no observable effect, retain the verification evidence rather than claiming no risk. If the project lacks an owner to perform the review, defer unattended upgrades until responsibility is assigned.

Maintenance reports should separate new defects from known limitations. A known limitation such as proxy source data remains visible until the source is replaced and validated; it should not be closed merely because the software still runs. For each release, publish a concise status of resolved issues, open risks and any claim restriction that users must retain.

4.8 Software Maintenance Plan

Maintenance covers code, input contracts, environment, model bundle, dashboard and documentation. It also protects research meaning because feature and label changes alter prediction interpretation. Assign a named maintainer before shared use.

For every scheduled review, capture date, reviewer, version and result. A proposed operational rhythm is to inspect failures and artifact completeness after each run, review dependencies and access before each release, and rehearse restore at a supervisor-approved interval. The interval must reflect the actual recovery need and storage policy; this document does not assert a cadence is in place.

Monitor data-contract drift as well as software drift. Changes in table columns, date coverage, missingness, normalized technology names or class balance can alter downstream outputs without a code change. Compare the latest manifest and quality report with the previous accepted run; escalate material deviations before interpreting model results.

Model maintenance should preserve training and evaluation context. Retain the preprocessing object, feature order, target definition, split metadata, model selection basis and explanation outputs with the model. Retraining should create a new run identifier and leave the previously accepted bundle available for comparison or restoration.

A dependency update is not automatically a harmless patch. Inspect release notes for changes to serialization, metric behavior, data parsing or explanation APIs; where uncertain, exercise loading and representative pipeline stages in a clean environment. Communicate material changes to users and revise the guide before distributing the new bundle.

4.8.1 Maintenance categories

Use a maintenance register with one record per issue or change: identifier, reported date, affected version, severity, owner, evidence, decision, target date and closure link. A small project can keep this in a controlled document, but the chosen location and editor must be agreed. This is a proposal, not a repository capability. Avoid recording credentials or restricted row-level data in the register.

A routine release review should check whether dependencies remain supportable, the run can reproduce its declared source mode, the model bundle loads and the dashboard has current artifacts. When any check fails, retain the prior accepted release and classify the failure before deciding whether to hotfix, revert or defer.

Preserve raw inputs according to the data owner's retention and access rules; do not assume that source CSV files may be retained indefinitely. Retain derived outputs only for an approved purpose and period. Document deletion or archival responsibilities separately if institutional data policy requires them.

Preventive work can include reviewing credential exposure and dependency advisories, but the presence of an audit package in a requirements file does not prove a scan has been run. Record the exact command and result if a security or dependency review is performed. Assign remediation priority with the relevant owner rather than implying automatic protection.

Table 4.10: Maintenance categories, triggers and verification

Category	Trigger/example	Minimum action	Owner
Corrective	Pipeline exception, broken view, missing artifact.	Capture traceback/run ID; fix, test, rerun; retain prior bundle.	Developer
Adaptive	Python/package, OS, database or container upgrade.	Clean build; run pipeline/dashboard checks; document compatibility.	Developer
Perfective	Usability/runtime change without semantic change.	Record impact; test affected task; update guide/release note.	Developer; user
Preventive	Dependency, credential, backup or retention issue.	Assign owner/deadline; verify remediation and recovery.	Supervisor; developer
Research-semantic	Source, feature, label, split, metric or SHAP change.	Supervisor review; version experiment; rerun comparison; retrain users.	Supervisor

4.8.2 Release, backup and monitoring

Retain source identifier (commit when available), dependency record, source mode, run time, manifest, model/preprocessing, evaluation and explanation outputs. The supplied checkout has no Git metadata, so no commit is claimed for this run. Do not overwrite the accepted bundle before the new one passes checks.

Set backup cadence from recovery needs; record location, retention, access and restore test. Acceptance requires a rehearsal. Monitor failed runs, missing outputs, source row/date changes and dependencies. Escalate unexpected class distribution or absent class before interpretation. No automated backup or production monitoring is evidenced.

4.8.3 Maintenance acceptance

Close work with request, affected component, risk, version, test evidence, artifact comparison, documentation update and rollback. Semantic changes need supervisor approval. Dependency-only changes still need compatibility checks for serialized models and dashboard loading.

The change record should connect request, review, implementation and release using one identifier. It should state whether the request changes data meaning, model behavior, displayed interpretation or operational access. Attach relevant test output and artifact comparisons, but redact secrets and personal data from shared records.

For high-impact changes, approval is conditional on a reviewable diff, source and target rationale, affected-user notice and a restore plan. If class support or metric eligibility changes, explain the consequence for historical comparisons. A technically successful pipeline is necessary evidence for many changes, but is not the approval itself.

After release, check whether the change achieved its purpose and whether new failures or user misunderstandings appeared. Record follow-up ownership and due date. If no user review is possible, identify that limitation and restrict claims about usability rather than marking the review complete.

A change request should be narrow enough to verify. Separate unrelated requests when they affect different contracts so that a failed test or reviewer objection has a clear disposition. Link each released change to the exact run it produced; do not use a shared filename as the only version identifier.

At closure, verify that the requester understands the outcome, conditions and any incomparable outputs. Mark a request deferred when evidence or approval is missing. The change log should distinguish requested, approved, implemented, verified and released states so that code availability is not mistaken for authorization.

Keep changes traceable to the academic plan when they affect research question, cohort, time window or evaluation method. The supervisor should decide whether the proposed change needs

a methodology revision or a note in the final discussion. A code review may identify implementation correctness, but it cannot alone establish that the research design remains appropriate.

4.9 Change Management Plan

Change control prevents unreviewed data, model, interpretation or runtime edits from silently changing result meaning. The lifecycle is proposed governance, not existing automation. Use the student developer, supervisor and operational user roles; add a second reviewer for high-impact research changes.

Classify requests by their highest plausible impact, not by estimated coding effort. A one-line threshold adjustment can change the target population, while a larger style change may leave model meaning intact. When impact is unclear, default to supervisor review until the affected contract has been identified.

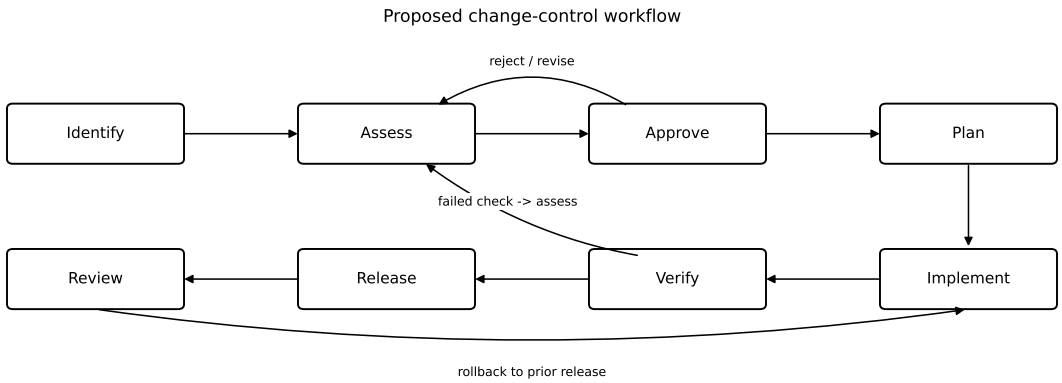
Assess affected components across the full flow: input definitions, ingestion, feature aggregation, labels, split, model selection, metrics, explanation join keys, dashboard wording, access and recovery. Include downstream reports and training material. This impact map prevents a source change from being treated as a loader-only patch when it changes what the displayed prediction represents.

Approval should record the decision, rationale, conditions and reviewer. Approval is not implied by code review or CI success. If rejected, retain the reason and any reusable work; if approved conditionally, make the conditions explicit in the release gate. Emergency containment may happen before full review, but subsequent documentation and verification remain required.

Release communication should identify the effective run and which earlier results are not directly comparable. Tell users what task or interpretation changed, what action they should take and where to report problems. For critical access issues, use the project's agreed incident route; do not expose sensitive details in general release notes.

4.9.1 Lifecycle

Figure 4.3: Proposed change-management workflow



The flow in Figure 4.3 starts with a request; rejection or failed verification returns for revision, and release failure can invoke rollback.

4.9.2 Change class and review

A change requester supplies the intended outcome and reason; the developer writes the impact assessment and plan; the supervisor approves research-semantic decisions; and an operational user reviews task effects. These roles are proposed. Where the same person must request and implement a change, require another reviewer for high-impact decisions and record the separation-of-duties limitation.

Verification depth should follow the affected contract. A wording-only change needs a proofread and meaning review. A loader change needs schema and ingestion checks. A feature or target change needs boundary examples, leakage review, retraining and a fresh evaluation interpretation. A dashboard change needs task walkthrough and empty-state checks. A release change needs bundle matching and rollback evidence.

Compare outputs only where comparison is valid. Changes to source coverage, time split, target population, feature meaning or class support can make historical score comparisons misleading. The review note should state which comparisons are retained, withdrawn or qualified; do not use a stable-looking score as a substitute for validating changed inputs.

Post-release review should happen after the agreed users have had time to perform the changed task. Check both technical outcomes and comprehension, especially when risk/confidence labels or source-mode notices changed. If the intended result was not achieved, reopen the request with findings and choose correction, rollback or further evaluation.

Table 4.11: Change class, verification and communication

Class	Examples	Review/release evidence	Owner
Low: editorial	Typo without meaning change.	Proofread; confirm technical claim unchanged.	Developer
Moderate: interface/runtime	Navigation or nonsemantic dependency patch.	Affected test/view check, package record, user notice as needed.	Developer; user
High: research semantic	Source, proxy, feature, label, split, metric or SHAP key.	Supervisor approval; pipeline; compare counts/eligible metrics; update method/training.	Supervisor
Critical: access/safety	Credential exposure, unintended access, misleading shared output.	Contain; preserve evidence; restore safe bundle; verify before re-enable.	Supervisor; developer

4.9.3 Request-to-close procedure

Request: assign ID, requester, reason and intended outcome. Assess: identify affected data, schema, features, targets, models, metrics, explanations, views and users; rate likelihood and consequence. Approve: supervisor decides semantic changes; user advises workflow changes. Plan tests, communication, backup and rollback trigger. Implement reviewably while retaining accepted bundle. Verify targeted tests and pipeline; inspect differences and warnings. Release only the approved coherent bundle. Review outcomes and feedback. Rejected/failed work returns to assessment; rollback restores the prior bundle and opens corrective action.

CI can verify repeatable technical behavior but cannot approve data rights, research definitions or comprehension. Link workflow evidence where available and name manual reviewers.

Retain the change decision with the release evidence so that the rationale survives personnel turnover. The record should identify approved scope, limitations, verification performed, outstanding risks, affected run identifiers and communication sent. If an approval condition is not met, the change remains unreleased even if its code is merged or available locally.

For a rejected request, explain the reason and any evidence needed for reconsideration. For an emergency correction, record containment first and complete the assessment and review afterwards. A rapid response can be necessary, but undocumented emergency edits should not become the new accepted baseline.

Where a release changes dashboard wording but not model output, still record the user-facing change because it can alter interpretation. Where a release changes only internal logging, record its operational effect and verify that logs remain useful without exposing private data. The change class should reflect the full user and research impact, not only the code component edited.

Release communication should accompany the bundle, not be sent as an untracked message. The note should identify the release and source mode, summarize changed behavior, list known limitations, state who approved promotion and tell users how to report an issue. When users may have saved earlier outputs, explain which run identifier is current and whether prior and new results can be compared. Retain the communication with the change record so future reviewers can recover the effective date and audience.

If an urgent correction is required, first contain the immediate impact and restore a safe state where necessary. Then complete the ordinary assessment, technical verification and supervisor review as soon as practical. Mark the change as emergency handling and preserve the timeline, reason, affected users and follow-up tasks. Emergency action changes the sequence of review; it does not remove the requirement to document or validate the resulting release.

4.9.4 Verification by affected contract

For ingestion changes, verify the source manifest, current table/column contract, representative parsed values and quality report; database-only behavior needs a database-backed check because local `CSV` smoke does not exercise it. For feature or label changes, review time windows, missing-window handling, target boundaries, class counts and leakage exclusions. For split, training or metric changes, retain split metadata, warnings, baselines and the reason each reported metric is eligible. These checks address different contracts and should not be collapsed into a single pipeline pass.

For explanation changes, verify that explanation rows still join to the intended technology and observation record and that dashboard wording does not imply causation. For interface changes, execute the affected user task, including missing-artifact states where relevant. For dependency

or container changes, rebuild the intended environment and load a coherent model bundle. Preserve the exact command or workflow result and identify checks that were not run.

A change is ready to release only when every affected contract has a result or a named, approved limitation. A missing check is not a pass. The supervisor can narrow an academic claim or defer use, but the decision and its effect on the final project must be recorded. This evidence map lets reviewers see why a particular test set was sufficient for that change.

Execution supplement: operator procedure and evidence handling

This procedure is a proposed operator guide for the checked-out repository. It is written to make the handover reproducible while keeping development mode separate from research use. The command examples assume the repository root is the current directory and a virtual environment has already been activated. They are not evidence that a production service, research warehouse, or operational support rota exists. Confirm that the checkout and intended source mode are correct before writing any generated files.

Begin by reviewing the project `README`, `requirements.txt`, .env.example, Dockerfile, `pipeline/run_pipeline.py` and `src/common.py`. The `README` describes PostgreSQL as the preferred full warehouse path and local CSVs as development inputs. Copy the environment template only into a local, ignored environment file if needed; never paste `DATABASE_URL`, `JWT` secrets, tokens, or passwords into a report, ticket, terminal capture, or sample record. A missing database setting intentionally selects the `CSV` development route, so make that route visible in the run record.

Create an isolated Python environment using the interpreter appropriate to the repository and install the declared dependencies. A proposed sequence is `python -m venv .venv`, activate the environment using the operating system's shell, then `python -m pip install -r requirements.txt`. The project `README` identifies Python 3.10+ while the Dockerfile uses Python 3.11; this is a compatibility boundary to reconcile, not proof that every Python minor version was tested. Record the actual interpreter and package resolution used for any accepted run.

Before execution, inspect the working tree and preserve any user changes. Run `git status --short` when Git metadata is present; this supplied checkout has no Git metadata, so that command may not establish a source revision. Record a source archive identifier or a checksum of the source bundle if a durable code identity is needed. Ensure that data inputs are approved for the selected use and that generated outputs will not overwrite an accepted bundle.

Run the pipeline from the repository root with `python pipeline/run_pipeline.py`. The orchestration in `pipeline/run_pipeline.py` executes ingestion, `EDA`, feature engineering, labelling, model training, evaluation and `SHAP` in order. It stops and records a failed stage on an exception. Capture the exit code and complete output, then inspect `outputs/data_sources.json`, `outputs/data_quality_report.json`,

`outputs/labelling_summary.json`, `outputs/model_comparison.csv` and the corresponding model and explanation files. An exit code of zero is necessary for a complete run, but it is not sufficient for research acceptance.

Check source provenance before interpreting any dashboard or model output. If the manifest says local development CSVs, retain the development-mode qualifier in every screenshot, table, demonstration and handover. Proxy-derived company adoption, metadata and mapping tables are not empirical observations. If a PostgreSQL run is intended, check that the manifest reports PostgreSQL and that all six expected tables were loaded; do not infer that database mode ran from the presence of a `DATABASE_URL` string alone.

Launch the repository dashboard with `python -m streamlit run `dashboard/app.py` --server.headless=true --server.port=8501`. Open the local address printed by Streamlit. Confirm that the dashboard's source notice, displayed run artifacts, empty states and caveats agree with the manifest and evaluation summary. Stop the service after the walkthrough. This is an operator procedure; a successful launch does not demonstrate authentication, deployment hardening, concurrent-user capacity or accessibility compliance.

If an artifact is missing, first identify which stage owns it and whether the run stopped before that stage. Compare file modification times with the run log, and do not combine old model files with a new feature matrix. If a file exists but appears stale, preserve it for diagnosis and run into a separate output location only if the application supports one; otherwise archive the accepted bundle before a new run. Record the run identifier and artifact inventory together.

If ingestion fails, distinguish a missing optional local input from a failed required PostgreSQL table. If features or labels fail, inspect the input schema and date coverage before changing thresholds. If training fails, inspect class support and preprocessing warnings. If evaluation omits a metric, retain the warning and explain why the test split cannot support it. If `SHAP` is slow, use a documented bounded development run only when the code exposes such a control; never conceal a cap or describe the resulting sample as full-population evidence.

If the dashboard displays no model, confirm that the expected model path and model name match the current run. If it displays no feature explanation, inspect the matching `SHAP` artifacts and technology keys. Clear Streamlit caches only as a diagnostic step and record that action. Never repair a mismatched artifact set by renaming a stale file to the expected name; rerun the owning stage or restore a coherent, versioned bundle.

At the end of the session, stop the dashboard, record the run command, environment summary, source mode, outcome, warnings, artifact paths and operator. Keep raw source files under their approved retention and access policy. Store the run record beside the derived artifacts, redact credentials, and state whether the results are development-only, research-reviewed, or accepted for a specific demonstration. These status labels are proposed governance controls.

Execution supplement: proposed test catalogue

The following catalogue is a proposed acceptance plan tied to repository modules, not a claim that every case has already been executed. The canonical development smoke run recorded earlier is separate evidence. For each case, save the command or fixture, expected result, observed result, environment and run identifier. A skipped test must be marked not run with a reason; it must not be counted as a pass. Where unit tests already cover a behavior, map that test to the case ID rather than duplicating code without cause.

Use `python -m pytest tests/ -v` for the repository test suite and `python pipeline/run_pipeline.py` for the end-to-end smoke test. These commands are grounded in the README. The test suite is not a substitute for a database-backed check, and the development pipeline is not a test of research-data validity. The operational user walkthrough is a separate manual check because automated tests cannot establish that the displayed wording is understood.

Table 4.12: Proposed test cases and observable acceptance criteria

ID / component	Input or setup	Expected result and pass condition	Evidence to retain
TP-ING-01 src/data_ingestion.py	Unset DATABASE_URL; repository CSV inputs available.	Development loader completes; source manifest explicitly identifies local development mode and proxies.	Command, exit status, manifest, source file inventory.
TP-ING-02 PostgreSQL loader	Configured test database containing six required tables.	All required tables load and manifest identifies PostgreSQL; no secret is exposed in logs.	Redacted configuration description, table inventory, manifest.
TP-ING-03 missing table	Test database omits one required table.	Ingestion fails with the missing-table condition; pipeline does not silently fall back to CSV.	Sanitized exception, database fixture schema.
TP-ING-04 quality report	Fixture includes null keys, short table and parseable date values.	Quality report records row counts, missingness, detected date range and applicable issues.	Fixture definition and JSON report.
TP-EDA-01 src/eda.py	Valid loaded development frames.	EDA stage completes and expected report/plot outputs are written to the configured output directory.	Run log and file listing.
TP-FEAT-01 src/feature_engineering.py	Small frame with technology, category and date candidates.	Feature matrix has a stable technology key and expected observation-period fields.	Input fixture, column list, output sample.
TP-FEAT-02 normalization	Two aliases known to normalize to one technology plus unrelated names.	Aliases resolve consistently; unrelated technologies remain distinct.	Fixture and normalized-key comparison.
TP-FEAT-03 constant numeric field	A feature is constant for all fixture rows.	Normalization follows implemented constant-column behavior without divide-by-zero output.	Fixture and resulting values.
TP-LABEL-01 threshold boundary	Recent average is positive; future/recent ratio equals the growth boundary.	Label is Growing at the inclusive configured threshold.	Boundary fixture, constants, label output.
TP-LABEL-02 decline boundary	Future/recent ratio equals the decline boundary.	Label is Declining at the inclusive configured threshold.	Boundary fixture and output.
TP-LABEL-03 missing target windows	One or both future/recent target columns are missing or nonnumeric.	Implemented fallback is observed and disclosed; no silent fabricated future observation is added.	Fixture, warning, label summary.
TP-LABEL-04 leakage audit	Inspect FEATURE_COLUMNS and TARGET_LEAKAGE_COLUMNS.	Future-window target fields are excluded from model inputs; assertion or review record has no overlap.	Code reference and feature-list output.
TP-TRAIN-01 chronological split	Feature matrix with ordered observation months and supported labels.	Training uses chronological ordering where configured; retained split metadata identifies boundary.	Split metadata and run log.
TP-TRAIN-02 absent class in a fold	Small fixture with class support absent in a training fold.	Training handles supported class encoding or reports a clear failure; no class meaning is remapped silently.	Fixture and per-model outcome.
TP-TRAIN-03 model artifacts	Complete eligible development matrix.	Each trained candidate artifact can be loaded with its matching preprocessing and class mapping.	Artifact inventory and load check.
TP-EVAL-01 metric eligibility	Test labels include fewer than all three classes.	ROC-AUC is reported as unavailable/not meaningful with a warning; it is not invented or silently set to a score.	Evaluation summary and warning.
TP-EVAL-02 confusion matrix labels	Predictions use the declared Growing, Stable, Declining encoding.	Confusion matrix retains the declared class order, including zero-support labels.	Matrix and encoding reference.

ID / component	Input or setup	Expected result and pass condition	Evidence to retain
TP-EVAL-03 baseline comparison	Candidate predictions and repository baselines are present.	Comparison output identifies candidate/baseline rows and selection rule; ties or ineligible results are documented.	Comparison CSV and selection record.
TP-SHAP-01 explanation alignment	Accepted model, feature order and matching input rows.	Explanation artifacts correspond to the current model/run and contain expected feature identifiers.	Model identifier, SHAP files, join check.
TP-SHAP-02 bounded smoke execution	Development run follows the documented sample cap, if enabled.	Run completes within the configured development path and record states the cap; no population-wide claim follows.	Command, cap value, output inventory.
TP-DASH-01 dashboard empty state	No trained model or prediction outputs available.	Dashboard handles missing artifacts with a visible empty/error state rather than a fabricated result.	Screenshot or AppTest assertion.
TP-DASH-02 coherent artifact set	Dashboard receives artifacts from one run identifier.	Displayed model, comparison, predictions and explanations identify the same coherent run.	Run manifest and walkthrough notes.
TP-DASH-03 proxy disclosure	Manifest marks development proxies.	User-facing view retains development-only notice wherever proxy outputs are interpreted.	Screenshot and wording review.
TP-OPS-01 end-to-end smoke	Local CSV mode; approved development inputs.	FR-01 through FR-11 complete or a real failure is captured; generated outputs are inventoried.	Full console output, exit status, manifest, file listing.
TP-OPS-02 rerun coherence	Repeat same declared development command after preserving prior output.	New run does not mix stale model or explanation files without a detectable run identity.	Before/after inventory and manifests.
TP-REL-01 restore rehearsal	Prior accepted bundle copied to controlled recovery location.	Operator restores the complete prior bundle and verifies model load and dashboard view.	Restore log, file checks, reviewer sign-off.

Worked example: one change request from identification to closure

The following scenario is illustrative. A reviewer notices that the dashboard's decline-risk ranking is difficult to interpret when a technology has a high predicted Declining probability but weak test-set support for that class. The request is to display a clear evidence caveat near the ranking and link users to the evaluation summary. It does not initially change features, labels, training, metric calculation or model outputs. The scenario demonstrates the proposed process; it does not assert that the repository already has this exact request or screen.

Identification: the operational user records a request ID, affected page, screenshot with data redacted, observed confusion and intended improvement. The requester states that the task is to understand the evidence behind the ranking, not to increase or decrease a technology's predicted score. The student developer records the current application version or source bundle identity, the source mode shown by the manifest and the exact location where the user encountered the wording.

Assessment: the developer traces the displayed risk value through the prediction artifact, model output, evaluation warning and dashboard component. The assessment checks whether the warning exists in `evaluation_summary.json` and whether the interface currently reads it. It identifies affected components as dashboard wording and artifact interpretation. The assessment explicitly excludes model retraining and target edits, unless code inspection shows

that the displayed rank is computed differently than assumed. The supervisor confirms that the proposed caveat does not overstate the statistical meaning of the score.

Decision: the supervisor classifies the initial request as a moderate interface change with a research-interpretation review condition. The operational user reviews the proposed wording in a representative development-mode screenshot. If the wording implies that a probability is calibrated or that decline is empirically established, the wording is rejected and revised. Approval names the exact text, relevant view, required empty-state behavior and condition that local-proxy mode remain visible.

Implementation: the developer makes the smallest readable change in the relevant dashboard module, adds or updates an automated assertion where practical, and leaves model files untouched. A review compares the diff against the request. The developer checks that the caveat derives from the evaluation output and that no stale static text masks a different run's warning. If the source does not contain a reliable machine-readable warning for the view, the developer records that gap instead of inventing a new metric.

Verification: the developer runs the targeted test and the dashboard test workflow, then walks through the affected page using a coherent artifact bundle. The test checks a warning-present case and a warning-absent or unavailable case. The reviewer checks text, visual placement, keyboard-readable order where the framework permits, and whether the link opens the matching evaluation summary. The local development source notice is confirmed. A full pipeline rerun is required only if the change affects generated artifacts or the change decision requires end-to-end evidence; the request record explains the chosen scope.

Release: the developer archives the previous accepted application bundle, records the source identity, writes a concise release note and confirms the approved wording. The note says that interpretation guidance changed and that prediction values did not change. The user receives the effective run or bundle identifier and the path for reporting confusion. The supervisor authorizes release after verifying the stated conditions; merging code alone is not recorded as approval.

Closure: after the agreed review interval, the operational user attempts the original interpretation task. The reviewer records whether the caveat was visible and whether the user can distinguish model output from evidence strength. If the task remains confusing, reopen the request with the specific failure. Closure retains the request, assessment, decision, diff reference, test evidence, screenshot, release note and follow-up owner. All sample identities and outcomes in this scenario are illustrative, not measured user-study results.

Table 4.13: Illustrative completed change request record

Field	Illustrative value
Request identifier / status	CR-ILL-004 / Closed after review; invented sample identifier.
Requester / proposed owners	Operational user / student developer; supervisor is approval authority for interpretation.
Reason / intended result	Users need the test-support caveat beside the decline-risk ranking so they do not interpret a weakly supported class as established evidence.
Impact assessment	Dashboard text and link only, subject to code trace. No model or target change is authorized by this request.
Source mode / limitations	Local development CSV with proxy-derived tables; demonstration only. This value is illustrative.
Verification required	Warning-present and warning-unavailable cases; coherent artifact walkthrough; source notice check; targeted tests.
Rollback condition / disposition	Restore previous dashboard bundle if warning is missing, mismatched to run, or wording overclaims metric meaning.
Decision / release evidence	Illustrative approval condition: retain proxy notice and show the matching evaluation warning. No actual approval is claimed.
Closure evidence	Illustrative walkthrough record and release note; replace with dated, real evidence before operational use.

Worked example: a complete conversion rehearsal

This rehearsal describes a proposed transition from the local CSV demonstration path to an approved PostgreSQL research warehouse. It is intentionally a rehearsal plan rather than a claim that a warehouse migration has occurred. The canonical loader in src/data_ingestion.py expects six tables: so_questions, dev_sentiment, fortune500_stacks, company_profiles, tech_metadata and question_company_mapping. A legacy schema diagram is insufficient; the rehearsal inspects the current loader and the target database directly.

Before rehearsal, the supervisor confirms data authorization, retention rules, source owner, approved extraction window and who may access row-level inputs. The developer prepares a redacted connection configuration through the approved secret store, a non-production database or isolated schema, and a backup or reproducible reload method. Record exact code identity, dependency resolution, schema version and source export identifier. Do not copy credentials into the acceptance record.

Inventory: for every table, compare required key fields from src/data_ingestion.py with live columns. Capture row count, column count, null profile, date coverage and technology-key uniqueness where meaningful. Identify missing fields and transformations before loading. The data steward decides whether nulls represent unknown, inapplicable, suppressed or extraction failure. Do not substitute a zero for an unknown value without an approved semantic rule.

Load: create the target using the approved schema process, load a controlled rehearsal snapshot, and keep source extracts immutable. Confirm timezone, date parsing, duplicate handling, character encoding and technology alias treatment. The loader currently reads the required tables

and writes a quality report and source manifest. Verify that those reports identify PostgreSQL and preserve per-table row totals. A successful `SELECT` is only a connectivity check, not a successful conversion.

Reconcile: compare source and target counts at table and key-grain levels, identify rows rejected or transformed, and review date coverage and missingness differences. Follow sampled records from each source to the loaded frame using approved, redacted keys. Recompute the feature matrix and inspect technology counts, month coverage and target-window availability. Explain differences caused by normalization, filtering or join rules. If the result cannot be reconciled, stop promotion and preserve the failing evidence.

Exercise: run the complete pipeline in the rehearsal environment, inspect each stage log and the run manifest, and validate model inputs against the approved feature contract. Check that future-window columns are not used as predictors, the chronological test split is respected where configured, and metric warnings remain visible. Verify the dashboard uses only artifacts from this database-backed run. Compare selected outputs to the agreed reference sample; large differences require a data or methodology review, not automatic acceptance.

Promote: the supervisor and data owner review reconciliation and limitations; the operational user completes the dashboard task walkthrough; the developer records the accepted database schema and restore plan. Promotion changes the effective source mode only after all conditions are met. If any gate fails, restore the prior development demonstration bundle or keep the research run isolated. A failed rehearsal is useful evidence and is not a completed conversion.

Close the rehearsal with a conversion manifest that includes table, source identifier, extract time, schema version, input count, loaded count, rejected count, transformations, quality findings, output run ID, reviewers and decision. Retain only information approved by the data owner. Record unresolved fields as blockers and identify who decides them. A signed-off rehearsal authorizes only the stated source snapshot and code identity; it does not automatically authorize future refreshes.

Table 4.14: Illustrative lineage manifest for a rehearsal

Manifest field	Illustrative entry / rule
Run / source mode	RUN-ILL-2026-04 / PostgreSQL rehearsal; invented values for form demonstration.
Source identity	Approved snapshot identifier recorded by data owner; never put secret connection strings here.
Schema contract	Six required table names and key columns checked against src/data_ingestion.py and the live schema.
Time / code identity	UTC extraction time and immutable source bundle or commit identifier; sample value must be replaced.
Counts / reconciliation	Per-table input and loaded counts, rejected rows, duplicate treatment, and signed reconciliation outcome.
Transformations	Alias normalization, date parsing, filters and derived fields, each linked to rule and reviewer.
Quality / caveats	Missingness and date-range findings; proxy fields explicitly identified; no unapproved personal data.
Generated outputs	Manifest, quality report, feature matrix, labels, model, evaluation, predictions, SHAP and logs for one run.
Decision / retention	Proposed approvers, promotion status, restore location, access owner and approved retention period.

Worked example: a complete retraining decision

A retraining decision should start with a trigger, not a calendar assumption. Illustrative triggers include a planned source refresh, a material schema or date-coverage change, a corrected technology alias, a label-rule revision, or a documented performance concern on an approved evaluation design. The owner first asks whether the observed change is a software defect, an input-contract shift, a research-method change or ordinary variation. These causes require different evidence and different approval.

The developer captures the current accepted run, the new source manifest, data-quality report, class counts, feature definitions and split boundaries. The reviewer compares row coverage and class support at the same grain and period where comparison is defensible. If source population or target definition changed, the resulting metric difference is not interpreted as a pure model improvement. If only code changed, verify that input identity and methodology are held constant before attributing a difference to training.

The supervisor reviews whether the target remains aligned with the research question, whether observation and future windows are still separated, whether all predictors are available at the prediction time, and whether the test period remains a valid future holdout. Check that the declared class ordering remains Growing, Stable, Declining and that model selection still follows the documented criterion. Any changed threshold or split definition receives explicit methodology approval before the candidate is trained.

Train the candidates and baseline under a new run identifier while preserving the previously accepted bundle. Inspect candidate failures, warnings, class support, preprocessing and serialized-model load behavior. Evaluation must show which metrics are eligible. A ranking change alone does not establish a better model: examine weighted F1 alongside class-specific

errors and the intended operational consequences. Where test support is inadequate, record the limitation and defer a superiority claim.

For the illustrative decision, suppose the new input snapshot contains a corrected category mapping but unchanged label rules. The recommendation is to retrain only if the mapping changes model features or the research protocol requires a refreshed candidate; otherwise perform a controlled feature comparison first. If retraining proceeds, compare the old and new runs on a common, eligible evaluation period where possible, document non-comparable periods, inspect SHAP alignment, and have the supervisor decide whether the new bundle replaces the old one.

Acceptance retains the trigger, decision rationale, code and data identities, feature and target definitions, split metadata, per-class support, eligible metrics, warnings, baseline comparison, explanation review, artifact inventory and reviewer disposition. Rejection preserves the candidate for analysis but leaves the previously accepted bundle active. The decision must not be reduced to a single score threshold unless an approved protocol defines that threshold and its consequence.

Risk register and mitigation timing

The risk score below is a planning device, not an empirical probability. Likelihood and impact use an ordinal one-to-five scale; the product is a prioritization aid rather than a calibrated estimate of loss. The owner reviews ratings at weekly checkpoints and after a material source or environment change. An issue that has occurred is managed as an incident or blocker, not left in the register as a hypothetical risk. The supervisor may change the scale before the project adopts it, but should preserve the original rating and rationale for auditability.

Proposed mitigation timing is tied to the seven-period baseline. High risks receive preventive action before dependent work begins. Risks involving data rights, proxy validity or target leakage cannot be accepted solely because the code runs. A schedule contingency is meaningful only when it has an owner, a trigger and a scope decision. If time is constrained, narrow claims or defer a feature rather than silently skipping a gate.

Table 4.15: Proposed implementation risks, ordinal score and response

Risk / L x I	Trigger and consequence	Mitigation and evidence	Timing / proposed owner
Proxy inputs mistaken for research evidence / 4 x 5	Local fallback manifest or dashboard notice omitted; results may be misrepresented.	Preserve manifest check, persistent source notice, research-mode gate and review of every exported figure.	P1 then every release / supervisor
Data authorization unresolved / 3 x 5	Warehouse access or retention basis is unclear; use could breach approval conditions.	Obtain written source-owner decision and minimize retained records before extraction.	Before P2 access / supervisor
Target leakage / 3 x 5	Future-window fields enter predictors; reported evaluation becomes invalid.	Audit feature and leakage lists, add assertions, inspect training matrix columns and reviewer sign-off.	P3 before training / developer
Insufficient class support / 4 x 4	Chronological test period lacks one or more classes; some metrics are unavailable.	Report class counts, retain warnings, avoid unsupported ROC-AUC interpretation and revise evaluation design only through approval.	P4-P5 / supervisor
Source schema drift / 3 x 4	Required column or type changes; loader or feature logic may fail or silently change.	Compare live schema with loader contract, test a representative extract and stop on missing required tables.	P2 and each refresh / developer
Artifact bundle mixed across runs / 3 x 4	Old model or SHAP file remains beside new predictions; dashboard presents incoherent analysis.	Use run identity and full inventory; isolate or archive output before rerun; verify joins and timestamps.	Each pipeline run / developer
Environment differs across machines / 3 x 3	Python/package differences alter parsing, serialization or APIs.	Record interpreter and resolved packages; rebuild clean environment and exercise load plus dashboard checks.	P1, P6 / developer
SHAP execution exceeds practical window / 3 x 3	Explanation stage is slow or memory-intensive; run ends incomplete.	Measure in development, use only documented sampling controls, record cap and retain a full research plan.	P4 and P6 / developer
Database connection or secret exposure / 2 x 5	Credentials appear in shell history, log or shared evidence.	Use approved secret handling, redact captures, rotate exposed credentials and test denied-access behavior.	Before P2 and each deployment / supervisor
Restore copy is incomplete / 2 x 5	Only model file is restored without matching preprocessing or evaluation context.	Define bundle manifest, test restore of the entire set, verify model load and dashboard presentation.	P6-P7 / developer
Schedule overrun / 3 x 3	Upstream gate slips and dependent packages are compressed without review.	Use dependency-aware status, reserve decision time in P7, reduce scope or extend baseline through change control.	Weekly / supervisor
User misreads probability or SHAP / 3 x 4	Prediction is treated as certainty, causation or investment advice.	Role-based walkthrough, plain-language caveats, example interpretation and comprehension check.	P6-P7 / operational user
Unsupported dependency / 2 x 4	Package upgrade or base image changes model loading or dashboard behavior.	Review release changes, rebuild in isolation, load serialized bundle and run affected checks.	Before release / developer
Insufficient handover coverage / 3 x 3	Only one person can run, explain or recover the system.	Have receiving maintainer perform run and restore steps; record unresolved access dependencies.	P7 / supervisor

Assumptions, limitations and threats to validity

The implementation plan assumes the current repository checkout is the version intended for review. It assumes that the student developer can run the local Python project and that required development dependencies are available. It does not assume a live PostgreSQL warehouse, approved research data, operational users, production deployment, backup service or scheduled retraining. Each assumption must be confirmed by the responsible owner before it becomes an acceptance claim.

The current local path is a development integration path. Repository documentation states that `stackexchange.csv` and `fortune.csv` supply observed community and company-profile inputs, while adoption-stack, metadata and question-company mapping tables can be deterministic development proxies. Survey usage measures are not direct sentiment. The outputs therefore demonstrate software flow but cannot by themselves establish enterprise adoption, sentiment, or real-world decline prediction performance.

The target compares recent and future average monthly activity using configured growth and decline ratio thresholds. This operationalizes a specific trajectory label, not an independently verified economic outcome. Changes to window length, minimum volume, threshold inclusivity, missing-window handling or legacy fallback can alter labels. Any such edit affects the research definition and should be treated as a methodology change with boundary tests and supervisor review.

Chronological validation reduces one form of temporal leakage when observation months are available, but does not remove every source of bias. The result can be affected by technology survival, changing platform participation, tags and naming conventions, missing older records, repeated entities across periods, source coverage, proxy construction and delayed data availability. A held-out period from the same platform does not establish generalization to a different community or enterprise population.

Model comparison is limited by the available rows, class support and chosen split. A test partition missing a class cannot support every multiclass discrimination statistic. Weighted metrics can be dominated by common classes and can obscure poor performance on Declining. Baseline comparisons, confusion matrices, class-specific counts, warning text and metric eligibility should therefore accompany any selected-model statement. Never fill an unavailable statistic with zero or a fabricated estimate.

Interpretability artifacts have their own limitations. `SHAP` values explain model output relative to a chosen background and feature representation; they are not causal effects or a guarantee that an input is correct. Feature correlation can redistribute attribution, and explanations can be unstable under retraining or changes in data. The dashboard should link an explanation to the correct model, row and run and should avoid language implying that a feature caused a technology's trajectory.

System-level validity is separate from model validity. A clean pipeline run establishes that the observed software stages completed on the declared inputs. It does not establish data authorization, external validity, user comprehension, performance under concurrent use, security hardening or long-term maintainability. Those questions need separate study designs, reviews or operational controls and should be left open when evidence is unavailable.

The effort and duration estimates are planning estimates based on package scope, not measured historical delivery rates. Actual effort depends on data approvals, schema readiness, compute behavior, academic review turnaround and user availability. The seven-week proposal assumes timely decisions and a single developer with supervisor and user review. The owner should revise schedule and scope together rather than disguising dependency delays as completed work.

Threats to this plan include incomplete documentation, drift between `README` claims and current code, generated artifacts left from earlier runs, ambiguous source identity, environment-dependent behavior and limited independent review. Mitigation is to inspect current code, verify artifact timestamps and manifests, capture exact commands, preserve a coherent run bundle and assign a reviewer. This document should be regenerated or amended when the underlying code contract changes materially.

The sample forms and worked examples are explicitly illustrative. Their IDs, dates, owners, approvals, decisions and closure outcomes are invented to show what a completed record could contain. They must not be copied into a project archive as evidence of actual approval or user testing. Replace them with dated records and real reviewers before using them to authorize a migration, retraining or release.

Acceptance evidence map and proposed handover packet

The handover packet should let a reviewer reconstruct what was run, why the outputs exist and what claims are permitted. Organize it by run identifier and include a compact index rather than relying on file modification times. Preserve the source mode and code identity next to the model bundle. Link the final evaluation to its test split and the explanation files to the same model and feature ordering. If any artifact is absent, identify the responsible stage and state whether the run is incomplete.

A proposed acceptance meeting uses the packet in sequence: first verify scope and data authorization; next verify source manifest and quality; then inspect target, features and leakage controls; next inspect split and evaluation eligibility; then load the selected model and explanation; finally perform the dashboard task and restore rehearsal. This order follows evidence dependencies. A later-stage success cannot cure an earlier contract failure, though the project may continue exploratory work with a clearly provisional label.

For the academic handover, the supervisor records accepted scope, limitations, unresolved risks and any claim restrictions. The student developer demonstrates the repeatable command and identifies the generated files. The operational user describes the task they can complete and the

interpretation caveats they retain. If the receiving maintainer cannot repeat a step, create a corrective action with owner and due date rather than recording an unqualified handover.

The packet is complete only for its stated purpose. A development demonstration packet can be complete while remaining unsuitable for empirical claims. A research acceptance packet requires approved source and method evidence. A deployment packet additionally requires operational, security, access, recovery and support decisions that this repository alone does not prove. Label the packet purpose on its cover or index so a reviewer does not assume broader acceptance.

Task-level schedule and dependency control

The package-level baseline is decomposed below into reviewable tasks A1-H4. These IDs are proposed planning identifiers and are not issue numbers already present in the repository. Estimates are person-hours for planning, include review and evidence capture, and do not imply measured productivity. The table assigns a proposed owner from the defined roles. Where a task requires approval, the developer prepares evidence but the supervisor or operational user retains the decision assigned to that role.

A predecessor is a minimum evidence dependency, not a prohibition on exploratory work. For example, model experiments can be used to discover runtime issues before the label contract is signed, but results remain exploratory and cannot pass the model acceptance gate. The release sequence respects accepted dependencies: source authorization and contract before conversion; target and feature approval before final training; evaluation review before dashboard promotion; and coherent bundle plus restore evidence before handover.

If a predecessor is delayed, mark the dependent task blocked or exploratory and record the reason. Do not relabel a downstream artifact as accepted simply to preserve the calendar. A schedule review may reduce scope, extend the seven-week proposal, or reassign work after checking role availability. It should not remove data authorization, leakage review or metric eligibility gates. The proposed dependency path remains visible so that academic reviewers can distinguish parallel preparation from valid promotion.

Table 4.16: Proposed A1-H4 task-level work breakdown, dependencies and estimates

ID / task	Predecessor	Completion evidence	Estimate / proposed owner
A1 Confirm question, scope and claim boundary	None	Signed scope note identifies intended user, research question, non-goals and development-versus-research claim boundary.	4 h / supervisor
A2 Inventory checkout and execution entry points	A1	File map names canonical pipeline, dashboard entry point, configuration and generated-output directories.	3 h / student developer
A3 Establish environment and source identity	A2	Interpreter, package resolution, checkout/archive identity, command and known setup issues recorded.	5 h / student developer
A4 Baseline review and gate setup	A1-A3	Weekly checkpoint sheet names owners, gate evidence, unresolved decisions and escalation route.	3 h / supervisor
B1 Confirm source authority and intended mode	A1	Written source-owner decision, permitted use, access boundary and retention constraints.	4 h / supervisor
B2 Inspect CSV inputs and manifest behavior	B1,A3	CSV file list, source manifest behavior and explicit proxy-field inventory.	5 h / student developer
B3 Check six-table PostgreSQL contract	B1	Live schema compared with TABLES and KEY_COLUMNS in src/data_ingestion.py.	6 h / student developer
B4 Reconcile quality and provenance rules	B2,B3	Quality report reviewed for counts, missingness, dates, keys, transformations and unresolved differences.	5 h / supervisor
C1 Document observation and prediction time	B4	Feature contract lists information available at prediction time and the observation-month meaning.	4 h / supervisor
C2 Review technology key normalization	C1	Alias examples and collision decisions checked against normalization behavior.	4 h / student developer
C3 Approve feature and leakage exclusions	C1,C2	FEATURE_COLUMNS and target-leakage fields reviewed; future-window predictors excluded.	5 h / supervisor
C4 Approve target window and boundaries	C1,B4	Thresholds, minimum-volume rule, missing-window handling and boundary fixtures approved.	5 h / supervisor
D1 Validate chronological split inputs	C3,C4	Observation-month availability, sort order and proposed split boundary recorded.	4 h / student developer
D2 Run candidate models and baselines	D1	Candidate outcomes, preprocessing warnings and artifact names tied to one run ID.	8 h / student developer
D3 Verify serialization and class mapping	D2	Selected bundle reloads; class encoder preserves Growing, Stable, Declining semantics.	4 h / student developer
D4 Review training evidence	D1-D3	Supervisor records whether candidates are provisional or accepted for evaluation.	3 h / supervisor
E1 Inspect held-out support and eligibility	D4	Per-class test counts and eligible/ineligible metrics documented from evaluation output.	4 h / student developer
E2 Compare candidate and baseline errors	E1	Confusion matrices, weighted metrics, baseline rows and limitations reviewed together.	5 h / supervisor
E3 Validate explanation/model alignment	D3,E1	SHAP input feature order, model identity, observation key and explanation files checked.	5 h / student developer
E4 Approve claim language and model status	E1-E3	Written decision states what evidence supports and explicitly does not support.	4 h / supervisor
F1 Inventory dashboard artifact contract	E4	Dashboard path and expected filenames traced to generator outputs and run identity.	4 h / student developer
F2 Walk through user tasks and empty states	F1	Operational user records task result, ambiguity, missing-artifact behavior and caveats.	5 h / operational user
F3 Correct verified interface gaps	F2	Reviewed code diff addresses agreed task without silently changing research meaning.	6 h / student developer

ID / task	Predecessor	Completion evidence	Estimate / proposed owner
F4 Recheck coherent run presentation	F3,E4	Model, predictions, evaluation and explanation displayed from same approved run.	3 h / operational user
G1 Select conversion strategy and scope	B4,F1	Decision names pilot, parallel, phased or direct path and explains reversibility.	3 h / supervisor
G2 Execute isolated rehearsal	G1,B4	Rehearsal run records table counts, quality report, feature generation and failure disposition.	7 h / student developer
G3 Reconcile and test rollback	G2	Reviewer follows restore instructions and validates previous coherent bundle.	5 h / student developer
G4 Approve promotion or record deferral	G2,G3	Data owner/supervisor decision names exact source snapshot and conditions.	3 h / supervisor
H1 Prepare role-specific runbook	A3,F2,G3	Steps cover install, run, launch, interpret, troubleshoot and recover with redacted examples.	5 h / student developer
H2 Deliver practice and assess task performance	H1,F2	Learner task record states pass, observed difficulty and follow-up; attendance alone is not acceptance.	4 h / operational user
H3 Assemble accepted evidence packet	E4,G4,H1	Index links code identity, manifest, run log, reports, tests, bundle, limitations and approvals.	4 h / student developer
H4 Conduct handover and close open actions	H2,H3	Receiving maintainer demonstrates rerun/recovery; unresolved items have owners and due dates.	4 h / supervisor

Seven-period execution calendar: proposed weekly review cards

These review cards make the one-week periods operational without turning them into calendar promises. A weekly period is five proposed working days for sequencing; it does not state a measured number of hours or require all work to be serial. Review slots are proposed and can be moved through schedule control. Every day concludes with a short evidence note: completed item, blocker, next owner and whether the downstream task is exploratory or accepted.

P1 ends only when the source-mode choice, environment, scope and gate owners are understood. P2 ends only when the source contract and data-use decision are reviewable. P3 closes the predictor and label definitions. P4 produces candidate artifacts but does not imply selection. P5 reviews metric eligibility and explanation alignment. P6 validates dashboard and conversion rehearsal. P7 closes acceptance, user practice and handover. If an external approval delays a period, the owner updates the baseline and tells the supervisor which dependent work is provisional.

Table 4.17: Proposed weekly focus, daily evidence and exit decision

Period / duration	Day 1-2 focus	Day 3-4 focus	Day 5 review and exit evidence
P1 / one week	Confirm question, scope, exclusions and role owners; inspect README and project layout.	Create isolated environment; record interpreter and package resolution; identify actual run commands.	Review scope note, environment record and initial risk list; authorize P2 source inventory or name blockers.
P2 / one week	Confirm source authority and retention; inventory CSVs and source-mode logic.	Inspect PostgreSQL table/key contracts, quality output, manifests and expected artifact locations.	Review source contract and unresolved mapping; approve only the declared rehearsal scope or defer access.
P3 / one week	Trace predictor time, observation periods, technology keys and future-window fields.	Test target boundaries and missing-window behavior; inspect leakage exclusions and class order.	Supervisor signs feature/target contract or records required methodology changes; training stays exploratory before this gate.
P4 / one week	Check split eligibility, sample sufficiency and training configuration; preserve prior bundle.	Run candidates and baselines; record warnings, runtime failures and serialized artifacts.	Review candidate inventory, run identity and split metadata; decide whether evidence is fit for evaluation.
P5 / one week	Review held-out class counts, confusion matrices, metric warnings and baseline comparison.	Check SHAP alignment, feature ordering, prediction keys and claim wording with supervisor.	Issue a model-status decision: accepted for specified demonstration, provisional, or deferred; do not claim unsupported generalization.
P6 / one week	Trace dashboard reads, source notice, empty states and coherent artifact set.	Run operator walkthrough and conversion rehearsal; check backup and restore of full bundle.	Record task outcomes, issues, screenshot evidence and conversion decision; release remains conditional on unresolved high risks.
P7 / one week	Deliver role-specific practice; check learner task performance and accessible wording.	Assemble handover packet, source/data limitations, open risks, restore instructions and support ownership.	Supervisor accepts stated handover scope, defers with named corrective actions, or extends the baseline; archive exact accepted evidence.

Pipeline stage dossiers and artifact acceptance

The pipeline contains ordered stages in `pipeline/run_pipeline.py`. The dossier below defines the proposed review question at each transition. Stage completion means that a function returned without exception; artifact acceptance also requires inspecting its inputs, outputs, provenance and relation to the run. If a stage writes to a shared directory, the operator inventories pre-existing files before rerunning and distinguishes current outputs from stale files.

FR-01 ingestion hands a dictionary of data frames to EDA and feature engineering. Verify the table keys expected by downstream code, source manifest mode, quality report and proxy flags. For PostgreSQL mode, confirm the six required tables loaded; for local mode, identify original CSV inputs and generated proxy tables. If the input mix differs from the declared source, stop and resolve the manifest before interpreting later stages.

FR-02 EDA should summarize the exact ingested frames. Compare its reported row and date counts with the quality report and note transformations or filters. A chart that renders is not proof that its axis, category labels or input population are correct. Review one representative output

per major signal, check missing values and ensure sample-based plots are labeled when applicable. Retain generated images with the run record instead of copying only a cropped screenshot.

FR-03 feature engineering produces technology-period predictors. The reviewer checks the grain, keys, observation period, feature ordering and derivation of each signal. Confirm that future activity used for labels is not part of the predictor list. For normalized technology names, compare representative raw values with canonical output and inspect collision cases. Retain the schema snapshot and a small approved sample, not restricted raw records, in the general handover packet.

FR-04 labelling appends trajectory labels using the configured future-window rule. Review the threshold constants and label summary, including counts for each declared class. Boundary fixtures should test equality at both thresholds and the minimum recent volume behavior. Missing target windows need explicit interpretation because the current implementation can use a fallback label rule; record when that branch is reachable and whether it is acceptable for the intended source.

FR-05 through FR-09 training must preserve the same feature set, target encoding, chronological boundary and run identity across candidates. Inspect candidate exceptions and warning text rather than treating an absent model file as a silent skip. Confirm preprocessing is serialized with the estimator where implemented. If a candidate cannot handle class support in a fold, record its disposition and verify that the selection logic compares only valid outcomes.

FR-10 evaluation compares candidates and baselines on the held-out partition. Review the actual test labels, support per class, confusion matrices, weighted metrics and warnings. In the checked local run, the chronological test subset lacked Declining and multiclass ROC-AUC was therefore not meaningful. Do not generalize that observation to a future database run; recalculate eligibility from each run's own split.

FR-11 SHAP writes explanations for a selected model and input set. Confirm model identity, feature order, row key and run ID before linking an explanation to a prediction. Global importance is an aggregate view; a local explanation is a decomposition for a particular row. Both are model-relative and sensitive to background and correlated inputs. Keep explanation language descriptive and avoid causal recommendations.

The dashboard is downstream of the pipeline and reads generated artifacts. Treat dashboard launch as a separate integration check: verify the intended output directory, source notice, model name, comparison file, prediction table and explanation artifacts. A green or successful page render cannot prove that all loaded files belong to one run. Use a deliberate missing-file case and verify a coherent failure/empty message rather than a plausible stale display.

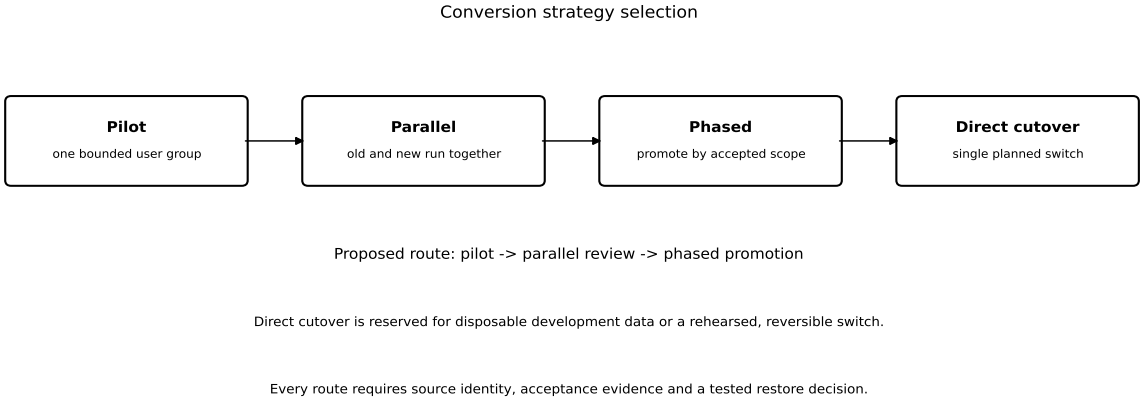
Table 4.18: Proposed artifact acceptance and stale-file control

Stage / output family	Required linkage	Review question	Disposition if missing or mismatched
FR-01 manifest and quality report	source mode, source snapshot, table counts	Does declared mode and source inventory match the actual inputs?	Stop downstream interpretation; correct source selection or record a failed ingestion.
FR-02 EDA reports and plots	same input frames and run timestamp	Are aggregation grain, period, labels and missingness understandable?	Mark EDA incomplete; do not substitute an older plot.
FR-03 feature matrix	source manifest, normalized keys, observation month	Are predictors available at the declared prediction time and leakage-free?	Reject matrix for training; correct feature contract and regenerate.
FR-04 labels and summary	matrix identity, target windows, threshold constants	Can each label be traced to the approved rule and support counts?	Hold training; inspect target columns and boundary handling.
FR-05-FR-09 model bundle	feature schema, encoder, split/run ID	Can estimator load with matching preprocessing and class meanings?	Keep prior accepted bundle; regenerate or restore complete candidate.
FR-10 comparison and evaluation	model predictions, held-out rows, class support	Are metrics eligible and warnings retained for this exact split?	Withhold model claim; repair evidence or state the limitation.
FR-11 explanation outputs	selected model, row keys, feature order	Does explanation identify the correct prediction and model?	Do not display explanation; regenerate from coherent artifacts.
Dashboard and release note	one bundle ID and current source notice	Do the shown values, caveats and links agree with the run packet?	Do not promote; restore a coherent prior bundle and open corrective action.

Conversion decision record and source reconciliation protocol

The conversion decision is a choice among pilot, parallel operation, phased promotion and direct cutover. The strategy depends on interruption cost, ability to isolate artifacts, source uncertainty, rollback confidence and who may review the converted data. This supplement's diagram gives a conceptual route; the accepted decision record should include the actual source, scope, date, authority and conditions. No one strategy is universally safer if the source permissions or restore method are unknown.

Figure 4.4: Proposed conversion strategy selection and safeguards



The proposed default path in Figure 4.4 combines a bounded pilot, parallel review and phased promotion. Direct cutover remains an option only when the source is disposable or the switch is rehearsed and reversible.

Source reconciliation protocol by table

Reconciliation should be performed per logical table, not only on the final feature matrix. The table names below follow the current loader contract. For each table, capture schema and row totals at a stable extraction snapshot, then compare loaded rows and accepted/rejected counts. Where records are transformed or joined, maintain an explicit rule and sample trace. The following checks are proposals; current local fallback frames may be derived proxies and will not satisfy a research-source reconciliation requirement.

For `so_questions`, review the question identifier, technology/tag field and creation date candidates. Check duplicate identifiers, invalid or future dates, tag normalization and whether deleted or closed content is included. The meaning of activity counts depends on inclusion rules; state them before interpreting monthly volume. The local file may contain a narrower extract than a full platform population, so provenance must state its extraction scope.

For `dev_sentiment`, confirm survey year, technology label, response denominator, weighting, suppression and the meaning of each score. A usage or admiration share is not an observed sentiment score. If a survey table is absent or represented by a development proxy, the manifest and dashboard must state that distinction. Do not bridge survey years into monthly observations without an approved aggregation rationale.

For `fortune500_stacks` and `company_profiles`, inspect company identity, technology aliases, sector definitions, reporting period and parent-company treatment. Derived mappings or deterministic proxies must not be described as verified adoption. Reconcile counts by company and technology, and document whether missing values mean no report, unknown or not applicable. Only the data owner can approve the interpretation of a source's coverage.

For `tech_metadata`, inspect canonical technology names, category changes, aliases, deprecated technologies and duplicate mappings. Keep a before/after mapping table under version control or the approved data governance location. A normalization change can merge previously separate series or split one series; quantify the effect on counts and labels before comparing results across runs.

For `question_company_mapping`, confirm join-key uniqueness, match rate, many-to-many behavior, effective dates and the basis for linking a question to an organization. Treat false joins as an interpretation risk because they can affect enterprise adoption features. Do not infer an organization identity from ambiguous text without an approved matching rule. Measure unmatched and multiply matched cases and state how they are handled.

When a source table has fewer than the code's 50-row diagnostic threshold, the loader records a quality issue; that warning is not automatically a rejection or proof of invalidity. The reviewer considers the table's role, expected population and source design. Likewise, the null warning threshold in the loader is a diagnostic rule, not a universal data acceptance threshold. The source owner and supervisor decide materiality for the research question.

Table 4.19: Proposed reconciliation fields by required source table

Logical table	Identity and grain	Required comparison	Question for review
`so_questions`	Question or activity record; technology/tag and date	Source count, loaded count, distinct IDs, date range, duplicate and date rejection counts.	Which records and platform period define the community activity signal?
`dev_sentiment`	Survey response aggregate by technology/year or defined unit	Year coverage, response denominator, missing/suppressed fields, name mapping.	Does the measure represent sentiment, usage, interest, or a derived proxy?
`fortune500_stacks`	Organization-technology observation at stated reporting period	Unique organizations, technology aliases, period, duplicates and unknown values.	What supports an adoption claim and what is only a development stand-in?
`company_profiles`	Organization profile at approved snapshot	Unique company IDs, sector coverage, parent treatment and null profile fields.	Can sector/company attributes be interpreted without overclaiming coverage?
`tech_metadata`	Canonical technology and category mapping	Unique canonical names, alias collisions, category changes, unmapped inputs.	Which mapping version governs comparison across periods and runs?
`question_company_mapping`	Question-to-company relation with linkage basis	Join rate, unique keys, multiple matches, unmatched rate and effective date.	Is the relationship evidence strong enough for the intended aggregate?

Operational troubleshooting playbook

Troubleshooting starts with preservation. Record the command, working directory, source mode, run ID, exit code and last completed pipeline stage. Copy logs to the run record before deleting or replacing outputs. Do not repeat a failing command against a shared output directory until the previous accepted artifacts have been inventoried. A retry can change timestamps or append logs and make the original failure harder to reconstruct.

For an import error, confirm the active interpreter and package resolution before changing versions. Compare `requirements.txt` with the installed environment, then run an isolated installation. Avoid one-off global installs that make the next operator's environment ambiguous. If the error names an optional database package during a local run, verify whether code attempted database mode or whether a stale configuration file supplied a URL.

For a database error, separate DNS/network refusal, authentication failure, missing table and query/load failure. Redact the connection string before saving output. Check the live table names and required columns with an authorized account. The loader raises a clear missing-table error when the connection is valid but required tables are absent; do not respond by silently substituting local CSV unless the run has been explicitly reclassified as development mode.

For a feature or label exception, preserve the source frame schema and the traceback, then inspect candidate column resolution, date parsing and target-window availability. Test the smallest synthetic fixture that reproduces the issue. Do not change label thresholds just to eliminate a failed assertion. Any target semantic change needs a supervisor decision, updated boundary tests and a new run identity.

For a training or evaluation warning, record whether it changes model eligibility or only describes a candidate. Examine class support before interpreting ROC-AUC or weighted metrics. If the test partition lacks a class, report the warning and consider a methodology review; do not alter the split after looking at candidate scores without recording the selection process. Retain the old accepted bundle while a new design is evaluated.

For a model-loading or dashboard error, check whether the estimator, feature order, encoder and prediction artifact belong to one run. Confirm that the dashboard resolves paths relative to the intended repository root. If an artifact is missing, test the empty state and avoid a fallback that shows an older result without a visible timestamp or run identity. Capture a screenshot only after source disclosure and bundle identity have been checked.

For a slow or interrupted SHAP stage, retain the observed start/end times and resource error if available. A bounded development sample can be a practical integration measure only when the cap is explicit in code or command and recorded in the run evidence. Do not describe a capped sample as complete. For research output, set a separate approved execution plan and verify that explanations correspond to the full intended population or clearly described sample.

After a correction, reproduce the failure case, run the targeted test, then run the smallest broader suite that covers affected contracts. A full pipeline run is appropriate for changes to ingestion, feature, label, training, evaluation or SHAP behavior. A documentation-only correction may need review and link checks but not a generated model rerun. The change record states why the evidence is sufficient and which checks remain unrun.

Role-based dashboard walkthrough scripts

Each walkthrough uses the same coherent run bundle but asks a role-specific question. These are proposed scripts for the operational user, supervisor and developer. The facilitator states the source mode and limitation before the task and avoids coaching during the first attempt. Record the task, prompt, observed completion, hesitation, question, corrective action and whether the user interpreted a model output as certainty. Do not retain personal or sensitive information in the notes without approval.

Operational-user script: ask the learner to locate a technology, state the displayed trajectory class and confidence, find the supporting evaluation limitation, open one explanation, and explain in their own words what it does not prove. Then ask them to identify the data source mode and explain what action to take if the view says outputs are missing. Success means the learner can complete the navigation and state the caveat without treating SHAP as causation or a risk label as advice.

Supervisor script: ask the reviewer to trace a displayed metric to the held-out split, inspect class support, distinguish unavailable from zero-valued metrics, and identify whether the source is development or research mode. The supervisor then checks that written claims match the evidence and decides whether the run is acceptable for its intended academic use. The script tests governance and interpretation, not model-building skill.

Developer script: ask the receiving maintainer to locate the pipeline entry point, state the chosen source mode, run the development smoke command, find the manifest and quality report, match model and explanation artifacts, and restore the prior bundle. The learner should describe how to stop when required database tables are missing and how to report a partial run. Success is a reproducible task with no exposed secret and no stale artifact substitution.

After each walkthrough, ask which label, metric or source note remained unclear. If a user completes the steps only after facilitator hints, record a partial outcome and adjust training or interface wording. A single successful walkthrough is not a usability study and should not be generalized to all users. If the intended participant group is unavailable, keep the usability question open rather than substituting the developer's own interpretation.

Table 4.20: Illustrative role-based walkthrough record fields

Record field	Operational user	Supervisor	Receiving maintainer
Task prompt	Find a trajectory and explain the source and evidence caveat.	Trace a metric to test support and decide if the claim is eligible.	Run pipeline, locate outputs and restore a coherent prior bundle.
Observable completion	Navigation, class/confidence interpretation, limitation and issue route.	Source mode, class support, metric status and claim disposition.	Correct command, run identity, manifest, artifact match and restore check.
Failure signal	Treats probability as certainty or explanation as causal.	Treats unavailable metric as zero or overlooks proxy source.	Uses stale artifacts, exposes credentials or cannot recover bundle.
Evidence	Dated observation note, prompt version, source mode and follow-up.	Review record and claim wording decision.	Sanitized console capture, inventory and recovery result.
Limit	One task does not establish population-wide usability.	Academic approval is limited to reviewed scope and evidence.	Successful local run does not establish production operations.

Release and restoration rehearsal detail

Before a release, define a bundle manifest with one run identifier and a list of required files. The list should be derived from actual downstream reads in the dashboard and pipeline rather than a generic model checklist. Include the model, compatible preprocessing or feature mapping, comparison/evaluation files, prediction output, explanation output, source manifest, quality report and logs when those artifacts are needed for the selected use. Do not require files that the current application does not produce without first changing and testing the application.

Copy the previously accepted bundle to a recovery location governed by the data retention policy. Compute checksums if the chosen storage process supports them, and record the restoration procedure. A copy operation without a restore test is not accepted backup evidence. The receiving maintainer restores into an isolated location, verifies the expected inventory and loads the model with the intended software environment. Then launch the dashboard against that bundle and confirm source mode and run identity remain visible.

Define rollback triggers before release. Examples include a source-mode mismatch, missing required artifact, model-load exception, wrong class order, explanation joined to an unintended row, critical user misunderstanding or an evaluation warning omitted from the display. The supervisor or designated release owner decides rollback authority and communication route. Preserve the failed release for diagnosis; restore the previous bundle without overwriting evidence.

After rollback, open corrective action with incident time, affected bundle, observed behavior, user impact, containment, restore evidence, root-cause hypothesis and next owner. Avoid claiming root cause before analysis. Notify users that earlier outputs are active again and identify which run they should use. The change record should close only after corrective action is verified or an explicit risk owner accepts the remaining limitation.

Table 4.21: Proposed release/restore decision gates

Gate	Evidence required	Decision owner	If gate fails
Identity	Source snapshot, code identity, environment and run ID agree across packet.	Student developer prepares; supervisor reviews.	Hold release; rebuild the manifest or repeat the run.
Completeness	All required artifacts for selected dashboard features are present and loadable.	Student developer.	Mark incomplete; do not mix prior files into the bundle.
Interpretation	Source mode, class order, eligible metrics and limitations are visible.	Supervisor and operational user.	Revise wording or defer the affected view.
Recovery	Prior coherent bundle restores and loads in isolated location.	Receiving maintainer; supervisor witnesses.	Do not promote until recovery works or risk is explicitly accepted.
Communication	Release ID, changes, known limitations and issue route recorded.	Release owner named in decision.	Keep release conditional; prepare comm unication before enablement.

Test execution worksheets: setup, action, oracle and evidence

The test catalogue gives IDs and acceptance conditions; this worksheet explains how to execute representative cases without confusing a test design with a completed test result. Before each test, capture the fixture identity, repository source identity, environment, command and initial output inventory. Use synthetic or approved de-identified data. Never write a test fixture that includes real credentials, unrestricted row-level research records or personal details in a shared report.

A test oracle must be observable and deterministic enough for review. For file-producing stages, compare expected names and schemas as well as process exit status. For labels, assert exact boundary behavior from constants rather than copying a score from a previous run. For model output, check class mapping and artifact identity rather than requiring a particular prediction from a mutable training dataset. For dashboard behavior, check user-visible content and artifact linkage, not only that Streamlit starts.

When a failure occurs, preserve the original fixture and failure trace. Record whether the issue is a regression, unsupported input, environment mismatch or an invalid test assumption. Update the test plan only through review when an acceptance expectation changes. A known failure stays open until corrected or formally accepted as a limitation for a clearly bounded use; it is not removed from the report simply because a later command succeeded.

Table 4.22: Proposed execution worksheet for representative contract tests

Case	Execution method	Pass oracle	Record and reviewer
TW-01 source mode	Start with `DATABASE_URL` explicitly unset in a clean process; run `` `python pipeline/run_pipeline.py` ``; inspect `outputs/data_sources.json`.	Manifest names local development mode and proxy provenance. Exit status alone is not the oracle.	Save redacted command, exit code and manifest; developer executes.
TW-02 database absence	Set no usable DB `URL` and verify local fallback is intentional; compare with a separate case using a reachable DB missing a required table.	Fallback is explicit only when configuration selects local mode; reachable-but-incomplete database fails visibly.	Retain sanitized logs and exact configuration conditions; developer reviews.
TW-03 required-table contract	On an isolated test schema, omit one table from the six-table loader contract and invoke ingestion.	Runtime reports missing required table; no partial research manifest is promoted.	Save schema fixture and exception text without connection secrets.
TW-04 missingness report	Pass a controlled frame with known nulls in key columns to the quality validator or a safe fixture pathway.	Reported percentages and issue entries match the fixture; date ranges ignore unparseable values.	Store fixture generator, expected assertions and `JSON` result.
TW-05 malformed dates	Provide valid, invalid and missing date examples through the source parser.	Valid range is reported from parseable values; invalid values remain visible as quality limitations.	Retain input schema and parsing outcome; do not expose raw protected records.
TW-06 technology alias	Use a small mapping fixture with known aliases, canonical forms and a deliberate collision.	Expected aliases resolve; collision is surfaced for review rather than silently conflating unrelated entities.	Record mapping version, examples and reviewer disposition.
TW-07 aggregation grain	Provide two technologies across multiple months with known activity totals.	Feature output has expected technology-month grain and no unintended row multiplication.	Compare expected and actual key sets plus totals.
TW-08 observation cutoff	Construct an observation window and later activity window with a clear temporal boundary.	Predictor rows use permitted observation period; target fields derive only from later period.	Save fixture dates and leakage assertion result.
TW-09 target growth edge	Set future/recent ratio equal to `GROWTH_RATIO_THRESHOLD` and just below it.	Equality and adjacent values follow the inclusive rule in `src/labelling.py`.	Record constants, fixture values and exact labels.
TW-10 target decline edge	Set future/recent ratio equal to `DECLINE_RATIO_THRESHOLD` and just above it.	Equality and adjacent values follow the inclusive rule; recent denominator floor is tested separately.	Keep boundary assertions with target definition review.
TW-11 zero or tiny recent mean	Use recent activity below `MIN_RECENT_AVG_VOLUME` with controlled future activity.	Ratio uses implemented minimum-volume floor; reviewer assesses whether this rule matches approved methodology.	Store both implementation expectation and supervisor decision.
TW-12 missing future window	Remove or invalidate future-window target values for one row.	Observed fallback path and resulting warning/label behavior are documented; no future value is imputed as observed.	Capture fixture, log and acceptability decision.
TW-13 leakage list	Compare `FEATURE_COLUMNS` to `TARGET_LEAKAGE_COLUMNS` and inspect the actual training matrix projection.	No target or future-window column reaches estimator fit inputs.	Save code references and assertion output.
TW-14 chronological order	Use known ordered observation months that straddle the configured train/test boundary.	No later observation is used to train the earlier evaluation; boundary is recorded in split metadata.	Retain rows by period counts and split boundary.

Case	Execution method	Pass oracle	Record and reviewer
TW-15 class mapping round trip	Encode then inverse-transform every declared class in the ordered label encoder.	Mapping is Growing=0, Stable=1, Declining=2 in both directions.	Keep assertion result with the model bundle record.
TW-16 rare-class fold	Create a small fold where a class is absent from one training subset.	Adapter or estimator behavior is explicit; predicted probabilities preserve declared class positions where supported.	Record candidate disposition and avoid requiring a training pass if unsupported.
TW-17 metric eligibility	Evaluate a test set with two of three labels and inspect warnings and JSON/CSV output.	ROC-AUC remains unavailable/not meaningful; no invented value is presented as a measured score.	Retain support counts and warnings with the comparison.
TW-18 baseline selection	Provide candidate and baseline rows with a known selection criterion.	Selected status follows the configured criterion; tie and missing-value handling is reviewed.	Save comparison table and selection rationale.
TW-19 serialized bundle	Load the saved candidate in a clean interpreter with the matching package environment.	Model and preprocessing load together and accept a row with the expected feature schema.	Record package resolution, model ID and load result.
TW-20 SHAP key alignment	Select known technology-period rows and compare explanation identifiers with prediction identifiers.	Keys and feature ordering match the intended run; no explanation is joined by row position alone where identifiers are available.	Retain join check and model identity.
TW-21 dashboard missing files	Temporarily point an isolated dashboard test to a directory without the model or prediction file.	View explains absence or empty state and does not show an older cached prediction as current.	Save AppTest assertion or redacted screenshot.
TW-22 proxy notice	Load a manifest marked development mode and inspect every view used in the walkthrough.	Notice remains visible near interpretation, including exported or screenshot context where applicable.	Keep reviewed screenshot set and wording decision.
TW-23 repeated run inventory	Run the smoke test twice after archiving the first output bundle.	Each run has traceable timestamps and artifacts; a mixed bundle is detectable and not silently promoted.	Store inventories and compare manifests.
TW-24 full restore	Restore the previous bundle to an isolated path and load it with the intended app.	Required inventory is present, model loads and dashboard reports the restored source/run identity.	Retain restore command, checksums if used and witness sign-off.

Feature contract review: from raw field to user-facing claim

For every feature, create a contract row with raw source field, logical table, key, time availability, cleaning rule, aggregation window, missingness behavior, output column, interpretation, known proxy status, downstream consumer and reviewer. If one source field contributes to more than one engineered feature, link every derivation. The contract is a research and engineering artifact; it should be versioned with the code or stored in the approved project record system. A prose name such as adoption score is inadequate if it hides a proxy or an unclear denominator.

Inspect units before normalizing. A count, rate, percentage, survey share and ordinal response cannot be treated as interchangeable numeric values. Min-max scaling depends on the observed sample range, so a new run can change every normalized value even if one technology is unchanged. For each normalized feature, record fit population, missing-value handling and whether scaling is recalculated on each period or across the full dataset. Scaling on future

observations may leak information if the evaluation design assumes prospective prediction.

Check aggregation windows for unequal exposure. A partial first or last month can appear to have low activity simply because fewer days were observed. If the code uses monthly bins, verify period completeness and timezone boundaries. If a technology has a short history, distinguish true decline from insufficient history. The operational threshold for inclusion should be justified against the research question and applied before looking at held-out outcomes where possible.

Check key integrity after each join. For each transformation, compare input and output row counts, distinct technology count, distinct period count, and duplicate key count. A many-to-many join can inflate activity without causing an exception. Where the transformation intentionally changes grain, state the expected multiplication or aggregation and assert the result. Use synthetic data with known totals to expose accidental Cartesian products.

For user-facing claims, trace a displayed value backward through dashboard, prediction artifact, model output, feature matrix and source contract. Write down where rounding, ranking, thresholding or class mapping occurs. The user should be able to distinguish model probability from a normalized feature score and from a risk category derived by application logic. If a label is not calibrated, do not call it a calibrated likelihood.

Review missing data by semantic category. Missing because the source did not collect a field differs from a measured zero, a suppressed survey response, a failed join, and a technology with no observation in that period. Document which cases are imputed, dropped or represented as missing. Run sensitivity checks only under a reviewed design and label their purpose; do not choose a favorable handling after observing test results without documenting the selection.

Retain a small, privacy-safe feature lineage example showing one source record or approved synthetic record, its normalized key, aggregate contribution, feature row and final dashboard label. This example helps reviewers verify that dates and joins are coherent. It must be synthetic or approved and should not disclose restricted user content. The example is explanatory evidence, not a population-wide validation result.

Table 4.23: Proposed feature lineage review template

Contract item	Entry to record	Review prompt
Source and authority	Logical table, extract/snapshot ID, source owner and approved purpose.	Is the source authorized for this analysis and is mode disclosed?
Key and grain	Entity key, time key, expected unique row grain and join relationships.	Could duplicates or many-to-many joins inflate a signal?
Time availability	Observation window, target window and latest field availability time.	Would this field be known at the intended prediction time?
Transform and units	Parsing, normalization, scaling, aggregation, unit and fit population.	Can an independent reviewer reproduce the derived value?
Missingness and proxy	Missing categories, imputation/drop rule, proxy status and limitations.	Is unknown distinct from zero and proxy distinct from observation?
Model/display mapping	Feature column, estimator position, output interpretation and dashboard label.	Does user language accurately describe what the number means?
Version and acceptance	Contract version, code/run identity, reviewer and decision.	Which earlier results become incomparable after a change?

Retraining review protocol and decision thresholds

A retraining recommendation should use triggers that can be observed in the run record. Potential triggers include a planned data refresh, a corrected source mapping, a dependency or model-code change, a change in class support, a failed artifact integrity check, or a supervisor-approved methodology revision. These are candidate triggers; their operational thresholds must be agreed for the intended project. The document does not propose an invented numeric drift threshold because the repository does not establish one.

First verify whether retraining is the right action. A source schema change calls for loader and reconciliation checks. A broken dashboard link calls for interface repair. A mislabeled class calls for target audit. A metric warning caused by a missing test class calls for an evaluation design decision, not necessarily retraining. Retraining can add work while leaving the root cause intact if the underlying issue is authorization, source quality or target validity.

If retraining is approved, preserve the prior bundle and create a candidate run with explicit data snapshot, feature contract, target thresholds, split boundary, random state, library environment and candidate set. A reproducibility claim requires all material sources of variation to be captured. If a model uses stochastic optimization, retain the random seed where applicable and note any nondeterministic library or hardware behavior that remains.

Compare old and new candidates using the same evaluation design where valid. Report both overall and per-class evidence, confusion patterns, test support, baselines and warning status. If source coverage changed, create a separate analysis of input-shift effects. If the test period or label definition changed, do not present a single score delta as a like-for-like improvement. State the comparison limitation in the decision record and dashboard release note.

A proposed retraining decision has three outcomes: promote for a named use, retain as candidate pending a specified evidence gap, or reject and keep the prior accepted bundle. Promotion

requires supervisor approval of research meaning and an operational user check of the changed interpretation. A candidate may be technically runnable but not acceptable. Record the exact intended use and expiration/review trigger rather than implying indefinite validity.

Where no drift monitoring exists, do not describe retraining as automatic. The owner can propose a manual review at each data refresh or release, but the chosen cadence must reflect data availability and project resourcing. If a future monitor is implemented, validate its input, threshold, alert route and response owner before relying on it. A chart or metric file by itself is not an operational monitoring system.

Training curriculum and skill checks by session

Training sessions should be scheduled after the system state is stable enough for the intended task. Each module has a prerequisite, demonstration, learner practice, observable check and follow-up. Use the same source-mode language and limitations in the training deck, dashboard and runbook. A learner who can repeat commands but cannot explain that local proxies are not research evidence has not met the interpretation outcome.

Session A, environment and source selection, is for the developer and receiving maintainer. Demonstrate checkout identity, environment creation, package installation, safe configuration and source manifest inspection. The learner practices identifying local versus PostgreSQL mode and finds the six-table loader contract. The check asks what to do when a database is reachable but a table is missing; the expected action is to stop and report, not silently switch modes.

Session B, pipeline and artifacts, teaches stage order and file provenance. Demonstrate one development-mode run from the repository root, then trace each log message to the stage and each artifact to its consumer. The learner inventories outputs, labels partial runs and identifies stale files. The check asks the learner to explain why a zero exit code does not authorize empirical claims and where to find the current source manifest.

Session C, labels and evaluation, is for the developer and supervisor. Review observation and future windows, threshold boundaries, class encoding, chronological split and metric warnings. The learner interprets a confusion matrix and explains why an absent class makes a statistic unavailable. The check uses a synthetic example rather than fabricated project results. The facilitator corrects any tendency to treat weighted performance as uniform class performance.

Session D, dashboard interpretation, is for operational users and reviewers. Demonstrate source notice, class output, confidence wording, risk ranking, evaluation caveat and SHAP explanation. The learner completes a task without coaching and explains limitations in ordinary language. The facilitator observes whether the learner treats association as causation or probability as certainty and records exact language that caused confusion.

Session E, release and recovery, is for the maintainer and supervisor. Demonstrate preserving an accepted bundle, creating a new run identity, checking required files, restoring a prior bundle and communicating a rollback. The learner performs the restore in an isolated location and

checks that dashboard source/run identity remains visible. The check fails if model, feature order, prediction and explanation files are mixed across runs.

Session F, change control and escalation, covers request classification, research-semantic approval, emergency containment, evidence capture and closure. The learner classifies sample changes and names who can approve each. The check emphasizes that a code review or CI pass cannot approve data rights or research definitions. Exercise a rejected change and a conditional approval so learners practice retaining rationale and open conditions.

Refresh training after a material interface, source, label, metric or recovery change. The owner decides whether a short delta session is sufficient or a full reassessment is needed. Record the changed material and task evidence. Training attendance can be documented, but effectiveness is measured only against the stated task and should not be generalized beyond the participants and session conditions.

Table 4.24: Proposed training sessions and role-specific checks

Session	Participants / prerequisite	Practice task	Observable outcome and follow-up
A. Environment and source	Developer/maintainer; checkout and access available.	Build environment, inspect config, distinguish local mode from a missing-table DB failure.	Correctly states mode and safe stop action; record setup gap.
B. Pipeline and artifacts	Developer/maintainer; source mode selected.	Run smoke pipeline, map stages to outputs, identify partial and stale files.	Finds manifest/log and labels incomplete run; update runbook if blocked.
C. Labels and evaluation	Developer/supervisor; feature contract available.	Use synthetic boundary case, read support and explain unavailable metric.	Preserves class semantics and metric caveat; revise methods note if unclear.
D. Dashboard interpretation	Operational user/reviewer; coherent bundle loaded.	Complete task, locate source notice and interpret one local explanation.	States limits without coaching; simplify wording if misunderstood.
E. Release and recovery	Maintainer/supervisor; previous bundle preserved.	Restore prior bundle and open dashboard against isolated copy.	Verifies full identity and load; recovery gap blocks handover.
F. Change control	All roles; change workflow reviewed.	Assess sample editorial, feature and access changes; classify approvals.	Names correct decision owner and evidence; retrain after semantic changes.

Schedule contingency decisions and escalation examples

A schedule contingency is a decision rule, not hidden slack. If P2 source approval is delayed, the developer can continue using the local development path for engineering checks while keeping research conversion blocked. The supervisor records that outputs remain development-only and decides whether the academic scope can proceed with that limitation. Do not describe a local `CSV` smoke run as a substitute for approved source evidence.

If P3 target approval is delayed, continue code inspection and synthetic boundary tests but do not freeze model selection or report final performance. Preserve any experiments as exploratory and re-run after the approved definition is recorded. If the target changes, update the feature contract, training material, test cases and release caveats; this is a cross-package change with schedule

impact.

If P4 training runtime exceeds the available window, first record the actual stage, elapsed time if measured, memory or process failure if observed, and whether earlier candidates completed. Consider a controlled bounded development smoke test only if the implementation supports it. Reducing sample size can validate integration but cannot substitute for the approved evaluation population. A revised compute plan needs supervisor review before claiming equivalent evidence.

If P5 shows missing class support or ineligible metrics, retain the output and warning. The supervisor can approve a revised evaluation strategy if it is methodologically justified, but the decision should not be chosen merely to obtain a more favorable score. If no defensible change is available, narrow the claim and state the limitation. Reallocate time from lower-priority interface polish to evidence interpretation only through an updated schedule note.

If P6 user review is unavailable, do not fabricate acceptance. Developer walkthroughs can catch obvious technical issues, but they are not evidence of operational comprehension. Mark the user check open, limit the intended use, and assign a participant/date decision. If a scheduled review is impossible before P7, the supervisor decides whether handover is conditional or deferred.

If P7 restore rehearsal fails, the system is not ready for unconditional handover. Keep the last accepted bundle and repository source safe, document the missing access or artifact, and create a recovery action with owner. If no recovery owner can be assigned, state that operational continuity is unresolved. A signed document cannot replace a demonstrated recovery route.

Escalation should identify the decision needed, latest safe date, downstream tasks affected, available options and claim impact. The supervisor can choose scope reduction, extension, reassignment or deferral. The developer supplies technical consequences and the operational user supplies workflow impact. Record the decision before implementing the chosen path so later reviewers can distinguish planned adaptation from undocumented drift.

Data-quality exception review and disposition

A quality warning should be triaged by the variable's role and the source design. Start by asking whether the warning is expected, unexpected, material to the feature or target, and recoverable from source. Record the raw diagnostic and the reviewer interpretation separately. The loader reports row count, column count, missingness, date ranges, technology count and selected key-column null percentages. Those diagnostics help direct review but are not comprehensive validation of research meaning.

For a low row count, compare the observed count with the expected extraction scope and table role. A table can legitimately contain fewer than 50 rows in a narrow snapshot, but it may be insufficient for a feature or class comparison. The code emits a `minimum_rows` issue; the project should not mechanically ignore it or mechanically reject it. The decision states expected coverage, consequence and whether a sensitivity check or additional source is required.

For nulls above the loader's key-column warning threshold, inspect whether missingness concentrates by period, technology, company, class or source. A global percentage can hide a subgroup with severe missingness. Do not infer missing-at-random from a summary. If the missingness affects a join, trace unmatched records and inspect the data-generation process with the source owner. Document exclusions and quantify their impact using approved aggregate outputs.

For unexpected date range or gaps, compare the range with extraction metadata and observation windows. Confirm parsing timezone and date coercion behavior; a malformed date can disappear from a min/max summary. Check whether the feature builder creates a complete month grid or only observed months. Avoid interpreting a missing month as zero activity unless the source contract supports that meaning.

For technology-key inconsistency, inspect raw and canonical forms, case, punctuation, version suffixes and aliases. A normalization change can improve consistency while also combining distinct technologies. Review a collision list with a domain-informed reviewer and preserve mapping changes. A feature count change after normalization should be explained before model metrics are compared.

For proxy-derived data, classify each table/column as observed, transformed, synthetic, deterministic proxy or unknown. The classification should flow from manifest to dashboard and report. Do not label the entire run research-grade if even a material input remains a proxy. A source-status matrix helps users see mixed provenance when some tables are observed and others derived.

A disposition record uses one of four proposed states: accepted for stated purpose, accepted with limitation, blocked pending evidence, or rejected. It names the affected stage, output, owner and next review point. A limitation accepted for a development demonstration may still block research promotion. Store the disposition with the run so the next operator does not have to reconstruct the decision from warnings alone.

Table 4.25: Proposed quality-warning triage record

Diagnostic	Investigation step	Disposition question	Required record
Short table / low count	Compare observed and expected source scope; inspect role and period coverage.	Is support adequate for this feature, label, or only a smoke test?	Expected count basis, observed count, owner and restricted-use status.
Null key threshold	Break down missingness by date, technology and join path; inspect source semantics.	Does missing mean unknown, not applicable, failed link or no observed event?	Aggregate profile, treatment rule, impact and reviewer.
Date parse/range	Check parse failures, timezone and extraction metadata against intended period.	Are periods complete and comparable? Are absent months true zeros?	Raw/parsed range, invalid count and approved period interpretation.
Name/key mismatch	Review alias map, collisions, unmatched keys and distinct counts.	Could normalization merge or split entities? Does join inflate rows?	Mapping version, collision decision and before/after aggregates.
Proxy source	Trace source manifest and generation rule to each affected field.	Can intended claim be made with this field, or must it remain demo-only?	Proxy inventory, disclosure location and claim restriction.

Development-to-research readiness and claim authorization

Development readiness and research readiness are separate decisions. The local pipeline can support integration work while some source tables remain proxy-derived. The `README` explicitly distinguishes local development outputs from a full research warehouse. A successful local run should therefore be accepted for software smoke testing only unless the supervisor documents a narrower approved use. Research promotion requires a new source decision and a new review of the data, target, split and claims; it is not a property inherited from an earlier development run.

The transition begins with a source inventory by table and field. Identify which values originate from observed community records, which are survey aggregates, which are derived, and which are deterministic development stand-ins. Record source owner, collection period, transformation and permitted purpose. If a table combines observed and proxy columns, disclose the mixture. A single run-level phrase such as research mode can conceal important differences if provenance is not granular enough.

Next, confirm whether the approved research source matches the loader and feature contracts. Schema compatibility alone is insufficient: a column can have the expected name but different semantics, time coverage or denominator. The supervisor and data owner review definitions, extraction dates and transformations. The developer tests the loader and reconciliation using a controlled snapshot before computing features. Any unresolved semantic mismatch blocks claims that depend on the field.

The research target should be approved in a signed methodology note before final training. That note states the unit of prediction, observation period, future outcome window, inclusion rules, class thresholds, missing-window policy and treatment of low-activity technologies. It states

why activity trajectory is a suitable operational target for the study and what it does not represent. Thresholds should not be tuned to obtain a desired distribution or score without an approved rationale.

The evaluation design then establishes what future period is held out, which observations can overlap by technology, how repeated entities are handled, and which metrics are eligible. A chronological split addresses ordering but may still place the same technology in both partitions. Whether that is appropriate depends on the research question: future-period prediction for known technologies differs from generalization to unseen technologies. Record this estimand explicitly before presenting the score.

Research promotion requires an evidence packet, not a configuration flag alone. The packet links source approval, source manifest, quality and reconciliation outputs, feature and target contract, split metadata, model comparison, warnings, explanations, user-facing wording, run identifier and supervisor disposition. If a component remains a proxy, the packet states whether the associated claim is removed, qualified or deferred. The decision should be specific to the snapshot and version reviewed.

A future data refresh does not automatically inherit acceptance from the first research snapshot. Recheck schema, counts, coverage, missingness, name mapping and class support. Material differences trigger re-evaluation of the target and comparison design. Keep previous accepted evidence for comparison and avoid overwriting files that are needed to reproduce a published or submitted result.

Table 4.26: Proposed readiness gates from local development to research use

Gate	Evidence required	Acceptable disposition	Block condition
R1 source authority	Owner, purpose, extraction scope, retention and access decision for every source.	Authorized snapshot with documented limitations.	Unknown rights, undisclosed proxy or prohibited retention.
R2 semantic fit	Field definitions and time/denominator meaning reviewed against the study question.	Definitions align or claims are explicitly narrowed.	Same-named field has unresolved meaning or coverage.
R3 conversion integrity	Schema check, row/key reconciliation, transformation record and quality triage.	Differences explained and accepted by reviewer.	Unreconciled counts, missing keys or unexplained join expansion.
R4 target validity	Prediction unit, windows, label boundaries and missingness policy approved.	Versioned target contract and boundary evidence.	Target rule selected after outcome inspection without rationale.
R5 evaluation validity	Estimand, temporal split, class support and metric eligibility documented.	Warnings retained and claims match available support.	Test leakage, absent class concealed or comparison is misrepresented.
R6 interpretation	Model, SHAP and dashboard language reviewed with source notice.	Use bounded to evidence and named audience.	Proxy outputs presented as observed adoption or causality.
R7 release and reproducibility	Run ID, code/data identity, coherent artifacts, restore and approval retained.	Accepted for the specific reviewed snapshot and purpose.	Bundle cannot be reproduced, loaded or restored.

Interpreting evaluation output without overclaiming

Accuracy answers the fraction of evaluated rows classified correctly, but does not show which class errors dominate. Weighted F1 combines class-specific precision and recall weighted by support, so a common class can contribute more to the aggregate. Precision and recall also need a declared averaging rule. Retain per-class counts and the confusion matrix so that a reader can see whether the minority trajectory classes are being missed.

A baseline contextualizes a candidate. A majority-class baseline shows what a trivial classifier can achieve under the observed class distribution; a momentum-rule baseline represents a simple domain heuristic in this repository. A candidate that exceeds a baseline on one aggregate measure is not automatically useful. Review the target, split, class errors, operational cost of false decline/growth signals and consistency with the intended decision before recommending use.

ROC-AUC summarizes ranking behavior under assumptions about class support and score outputs. The evaluation implementation attempts multiclass ROC-AUC only when all three encoded classes are present in the test labels. If a class is absent, the output warning is substantive. Do not replace it with a binary reinterpretation or infer that the metric is zero. If the study changes to a different AUC formulation, define and validate that formulation in a separate methodology decision.

A chronological holdout represents a future time period only if dates are correctly parsed, sorted and separated before any operation that learns from the full dataset. Preprocessing, scaling, imputation, feature selection and model tuning should be fit only on the training data where the design calls for prospective validation. Review actual pipeline boundaries rather than assuming a chronological label in the report guarantees the absence of all temporal leakage.

Repeated technologies across the train and test periods can be appropriate for forecasting future trajectories of known technologies, but it does not evaluate generalization to unseen technologies. Conversely, a grouped split may answer a different question and reduce available data. The project should state which population the test supports. A single evaluation cannot automatically answer both questions.

Hyperparameter search or model selection should not use the final held-out test repeatedly as a tuning oracle. Keep the test partition for the declared final comparison or explicitly revise the validation design. Preserve the candidate-selection rule and any manual decisions. The repository implementation includes multiple candidate models and baselines; the plan should retain which selection procedure actually ran rather than infer it from the best-model filename.

Prediction probabilities should not be called calibrated unless calibration has been evaluated under an appropriate design. Confidence displays derived from maximum class probability can appear precise even when calibration and distribution shift are unknown. Explain the display's computation and limitations. Avoid mapping raw model probability directly to business action

thresholds without an approved decision analysis.

A difference between two model scores needs uncertainty and comparability context. If the test set is small or shared across candidates, a small numeric difference may not support a strong superiority statement. If input population or label definition changed, even a large difference may reflect a changed task. This plan does not invent confidence intervals or significance tests; the supervisor should select an appropriate inferential method if a comparative claim is required.

Evaluation acceptance should state the permitted sentence. For example, a development-mode run can support that the pipeline generated a comparison file and emitted a warning for a missing test class. It cannot support an empirical claim about future adoption or real-world decline. A research-mode packet may support a narrower statement about the named sample and period after source and method review; it still does not establish universal prediction performance.

Software and hardware capacity review without unsupported minima

The repository documents Python 3.10+ in its `README` and uses Python 3.11 in the Dockerfile. Treat these as declared compatibility references to validate, not a measured range. The developer records the actual interpreter, operating system, package resolution, architecture and relevant environment variables for each accepted run. If two supported environments are claimed, exercise both or narrow the claim to the environment that was actually verified.

The software baseline includes the Python pipeline and dashboard dependencies in `requirements.txt`. PostgreSQL is needed only for the configured warehouse route; local `CSV` development does not require a database service. Docker is an optional packaging route when container delivery is chosen. The chapter generator has separate ReportLab, matplotlib and pypdf requirements that are not necessarily needed by a TechPulse operator. This distinction prevents unnecessary installation work and reduces ambiguous setup paths.

No hardware minimum is asserted because the repository does not provide benchmark evidence sufficient to establish one. Instead, during P1 and P4, measure the actual run on the intended environment: wall time per stage, peak memory if available, data row counts, explanation sample size, storage generated and whether other work competes for resources. Record measurement method and machine description. A single development result is a planning observation, not a general service guarantee.

Estimate capacity from the expected research extract only after its approximate size and retention are known. Identify which stages hold multiple frames in memory, whether `SHAP` computation scales with rows and features, and whether plots or model artifacts are retained. Use a representative authorized sample to test runtime behavior. If the sample is bounded, record its selection rule and do not extrapolate a precise production capacity without validation.

For container use, inspect the Dockerfile base image, copied files, working directory, exposed ports, environment configuration and entry point. Confirm that secrets are not baked into an image and that generated outputs have a deliberate persistence location. Build success is a

packaging check; run the pipeline and dashboard in the intended container mode to establish runtime behavior. Verify image provenance and update process before a shared deployment.

For workstation operation, maintain enough free storage for current and previous coherent output bundles, logs and approved inputs. The required retention amount depends on the actual data and policy; the plan does not assert a numeric capacity. Avoid writing large generated files to ephemeral folders if they are needed for review. Conversely, do not retain sensitive data indefinitely simply because disk space is available.

Capacity sign-off records the tested environment and observed workload, user count if any, data scale, runtime, memory/storage measurement source and unresolved bottleneck. It states explicitly whether it covers a one-person local demonstration, a research batch run or a shared service. If no concurrent-user or load test was performed, the handover says so and does not imply operational scale.

Maintenance scenario cards and service ownership

Maintenance ownership starts with the person who can authorize action, the person who can implement it and the person affected by the outcome. In a small academic project, roles may overlap, but the record should still name the decision-maker. The following scenarios show how the maintenance categories produce different evidence. They are proposed procedures, not reports of incidents that have occurred in TechPulse.

Scenario: a dependency update changes model deserialization behavior. The maintainer creates an isolated environment, reviews package changes, loads the previously accepted estimator with matching preprocessing and runs a representative prediction fixture. The dashboard is then opened against the same bundle. If loading fails, the update is not promoted; restore the previous package environment and preserve the exception. The supervisor decides whether the environment change affects method reproducibility.

Scenario: a source adds a new technology alias. The data steward or supervisor reviews the alias and canonical mapping, the developer creates a mapping fixture, and the feature counts are compared before and after normalization. If the alias merges distinct technologies, the change is rejected or the mapping rule is revised. If it is accepted, generate a new run identity and state whether historical aggregates are recomputed.

Scenario: a monthly source refresh creates a date gap. The developer compares the refreshed source range with prior manifest and extraction metadata, tests whether missing months are absent observations or failed extraction, and checks the feature grid. The supervisor decides whether the gap invalidates the affected period. Do not fill a month with zero activity unless that semantic decision is approved and implemented consistently.

Scenario: a user reports an explanation that appears inconsistent with a prediction. Preserve the exact run, technology-period key, model and feature row. Verify the join, feature order, class output and SHAP calculation. If artifacts belong to different runs, withdraw the explanation and

correct bundle integrity. If the link is sound, review the explanation semantics and dashboard wording; do not characterize a model-relative attribution as a causal contradiction.

Scenario: a generated report contains proxy data without its notice. Contain distribution of the report, locate all affected exports and screenshots, restore or regenerate from the correct manifest, and inform the supervisor and known recipients through the agreed route. The corrective change should verify that disclosure appears in every relevant view and the build process. Closing the incident requires evidence of corrected outputs, not only a code edit.

Scenario: the receiving maintainer loses access to a required account. Do not share personal credentials. The supervisor contacts the approved account owner, assesses whether access can be reassigned, and records the period during which the pipeline cannot use the source. Local development may continue if appropriate, but the run is clearly reclassified. The handover remains conditional until access and recovery responsibilities are confirmed.

Scenario: an accepted candidate has a new class-support warning after refresh. The developer checks source coverage and target counts, distinguishes a real population shift from extraction failure, and produces a new evaluation summary. The supervisor reviews whether the previous claim still applies. The model is not automatically retrained if the new warning is caused by the evaluation sample; the decision may be to defer use or gather more approved data.

Scenario: a restore test fails because the model exists but supporting files do not. Keep the failed restore copy isolated, compare it with the bundle manifest and identify the missing artifact owner. Reconstruct from the original run only if the exact source and code identity can be recovered. If not, treat the bundle as unreproducible and choose a new accepted run with a clear limitation. Do not backfill missing files from a different run.

Table 4.27: Proposed maintenance scenario response and closure evidence

Scenario	Containment / first check	Verification before closure	Owner decision
Dependency update	Isolate environment; preserve old resolution and bundle.	Model load, representative prediction, dashboard, package record.	Developer implements; supervisor reviews reproducibility impact.
Alias mapping change	Hold promotion; review canonical key and collisions.	Fixture, before/after aggregates, label/run comparability note.	Supervisor approves semantic mapping; user checks display.
Date gap	Preserve extract; compare manifest and date parsing.	Expected coverage, month grid and approved missing-period meaning.	Data owner clarifies source; supervisor limits affected claim.
Explanation mismatch	Withdraw suspect view if bundle linkage is uncertain.	Model/run/key/feature order/SHAP join verified end to end.	Developer repairs; supervisor reviews interpretation wording.
Proxy notice omitted	Stop report circulation and identify affected outputs.	All relevant views and exports regenerated with accurate notice.	Supervisor approves corrected communication and reuse.
Access owner unavailable	Do not share credentials; classify affected source path.	Approved reassignment, or run clearly limited to another mode.	Supervisor names access owner and any scope consequence.
Class-support change	Preserve new evaluation and compare source coverage.	Counts, metric eligibility, warnings and use decision documented.	Supervisor decides retrain, defer, or narrow claim.
Restore incomplete	Keep copy isolated; compare with manifest and source identity.	Complete same-run bundle loads and dashboard identifies it correctly.	Maintainer resolves; handover remains conditional until pass.

Review meeting protocol and decision record

A weekly review should begin with the last accepted baseline and the change since that review. The developer shows the exact evidence path and source mode; the supervisor identifies decisions needed; the operational user reports task impact when relevant. A meeting note records decisions and open actions, not a transcript. Each action has an owner, due date or next review point, and evidence needed to close it.

The proposed P1 review confirms scope, intended claim and role availability. P2 reviews data authority, source inventory and schema. P3 reviews feature, target and leakage decisions. P4 reviews candidate generation and split metadata. P5 reviews metric eligibility, baselines, warnings and explanation alignment. P6 reviews dashboard tasks, conversion rehearsal and restore. P7 reviews training, residual risk and handover. If evidence is not ready, the meeting records a deferral and the downstream impact rather than forcing an acceptance.

Separate fact, interpretation and decision in the note. Fact: the manifest identifies local development mode. Interpretation: this run is not suitable for an enterprise adoption claim. Decision: continue pipeline integration work but withhold research promotion. This separation prevents a recommendation from being mistaken for an observed result and lets a later reviewer challenge the reasoning without losing the underlying evidence.

A decision record should include the question, options considered, evidence reviewed, assumptions, consequence of each option, selected action, approver, conditions, communication and revisit trigger. For a high-impact research-semantic change, note whether methodology or study scope needs amendment. For an access or safety event, record containment and the incident route separately from ordinary schedule status.

Meeting participants should not approve evidence they have not reviewed. If a reviewer was absent or a required file was not available, record conditional status and the missing item. Approval applies only to the version and scope cited in the decision. A changed source snapshot, code identity or label definition may invalidate an earlier approval and trigger a new review.

Open actions are carried forward until evidence closes them or the supervisor explicitly accepts or defers the risk. Closed actions link to artifacts and state the closure criterion. An action should not be closed because time has elapsed or because dependent work continued. At P7, unresolved high-impact actions are listed as handover restrictions with an owner and decision date.

Table 4.28: Proposed implementation review decision record

Field	Required content	Quality check
Decision ID and scope	Unique record ID, period, affected package, code/source/run identity.	Could a future reviewer identify exactly what this approval covers?
Question and options	Decision needed and feasible alternatives, including defer/reject.	Is a real choice presented rather than an assumed approval?
Evidence reviewed	Manifest, tests, reports, warnings, screenshots, restore and relevant code paths.	Are observations distinguishable from recommendations?
Assumptions and consequence	Unverified assumptions and impact if they are false.	Are limitations visible and bounded?
Decision and authority	Approve, conditional, defer or reject; named role and rationale.	Does approver have the role assigned for this decision?
Conditions and actions	Owner, evidence, due/next-review date, communication and rollback trigger.	Can each condition be objectively closed?
Revisit trigger	Source/code/method change, failed test, user issue or scheduled review.	Will changed circumstances invalidate or reopen acceptance?

Output review and publication safeguards

Before copying an artifact into a report or presentation, identify its source run and verify that the caption states the population, period, source mode and limitation needed for interpretation. A plot title can omit detail if the surrounding caption supplies it. Do not crop away a development notice, class-support warning, axis unit or data period. If an output cannot carry a full explanation, place a clear reference adjacent to it and ensure the exported version preserves that reference.

Check every table for units, denominator, date range, sort order, missing values and rounding. A percentage without a denominator is ambiguous. A count without an extraction period can be mistaken for a current total. A ranking without a tie rule can appear deterministic when it is not. Record generation code and run identity so a reviewer can recreate the displayed order.

Check every chart for meaningful axis scales, labels, legend, sample qualifier and accessible contrast. A truncated y-axis can amplify differences; a missing class can make a comparison appear complete. Figure metadata and alt text should express the same source and caveat as the visible caption. For a PDF, rasterize pages containing charts and inspect clipping at the final page size rather than assuming vector embedding guarantees a usable layout.

Check narrative claims against the exact evidence. Replace verbs such as proves, guarantees or causes when the design supports association or software execution only. State if a result is a smoke-test outcome, a descriptive summary, a held-out estimate or a user observation. Keep the quantifier aligned with the sample and source. If output is illustrative, label it as illustrative in the figure/table caption, not only in a separate distant paragraph.

When sharing an output externally, confirm data access rules, licensing and privacy review. Aggregate data can still reveal sensitive details in small categories. Suppress or combine categories only under a documented policy and check that transformations do not distort the research result. The report should not include secrets, personal details, raw unrestricted rows or internal paths that reveal credentials.

Preserve the exact delivered artifact and its text extraction or checksum in the handover packet. If a PDF is regenerated after a correction, identify the new version and retire earlier copies using the approved process. Verify page count, captions, bookmarks, TOC, footer, table wrapping and figures after the final build. A successful script run does not replace this visual and structural inspection.

Worked technical example: trace a trajectory label at its boundaries

This calculation is an illustrative rule walkthrough using the constants in src/labelling.py; it is not an observed TechPulse result. Let a technology-period row have recent average monthly volume R and future average monthly volume F. The implementation divides F by the larger of R and MIN_RECENT_AVG_VOLUME, then compares the ratio with GROWTH_RATIO_THRESHOLD and DECLINE_RATIO_THRESHOLD. The current constants are 1.20, 0.80 and 1.0 respectively. Any academic change to them alters the target definition and needs approval.

For a synthetic row with R=5 and F=6, the ratio is 6/5=1.20, which meets the inclusive growth boundary and is labeled Growing. With R=5 and F=4, the ratio is 0.80, meets the inclusive decline boundary and is labeled Declining. A ratio strictly between 0.80 and 1.20 is labeled Stable. These values illustrate arithmetic only; they are not records, counts or performance metrics from the project's data.

The minimum-volume floor changes the denominator when recent volume is below 1.0. For example, if R is 0.5, the code uses 1.0 as the denominator rather than 0.5. A future volume of 1.2 then reaches the growth boundary under the implemented rule. This protects a very small denominator from producing a large ratio, but it also encodes a substantive threshold that must

be reviewed against the research question. The plan does not claim that the floor is empirically optimal.

If both future-window and recent-window columns exist but either value is missing or nonnumeric, the current helper returns Stable for that row. That behavior is a default label, not a measurement that the technology is empirically stable. The reviewer should quantify how often this branch is taken, determine why the values are missing and decide whether those rows should be excluded or retained under an approved policy. The current run's label summary should not hide this distinction if missing windows are material.

If the two future/recent columns are absent altogether, the implementation logs a warning and falls back to the legacy `growth_momentum_index` rule: values above 0.70 become Growing, below 0.40 become Declining, and the remaining interval is Stable. The reviewer should verify whether this branch is reachable in the intended pipeline and whether legacy values are comparable with future-window labels. A missing target contract must not cause the legacy path to be treated as equivalent by default.

Boundary tests should cover exact equality and values immediately on either side of both thresholds, a zero recent average, a value below the minimum floor, missing future data, missing recent data, nonnumeric strings, absent columns and an empty feature frame. Each case should assert the expected label and whether a warning is required. The supervisor reviews the test expectations against the signed target definition; the developer does not select expected labels by observing model scores.

A label lineage record stores the input row key, observation period, recent and future window definitions, raw/summarized values where permitted, ratio calculation, threshold version, output class and treatment of missingness. For a synthetic example, replace raw row identifiers with invented keys. For an approved empirical audit, restrict the record to the least data needed and follow the source owner's access controls. This lineage makes the target reviewable without distributing sensitive raw events.

Table 4.29: Illustrative target-rule boundary examples; no project metrics

Scenario	Recent average R	Future average F	Rule result	Review significance
Growth equality example	5.0 illustrative	6.0 illustrative	F/max(R,1.0)=1.20; Growing at inclusive threshold.	Unit test demonstrates equality behavior; values are synthetic.
Decline equality example	5.0 illustrative	4.0 illustrative	F/max(R,1.0)=0.80; Declining at inclusive threshold.	Unit test demonstrates equality behavior; values are synthetic.
Interior example	5.0 illustrative	5.0 illustrative	Ratio 1.00; Stable under configured interval.	Does not mean missing values should be considered stable.
Small denominator example	0.5 illustrative	1.2 illustrative	Denominator floor is 1.0; ratio 1.20; Growing under current code.	Shows why minimum-volume assumption needs methodology review.
Missing target value	Illustrative missing	4.0 illustrative	Current implementation returns Stable when target columns exist but either value is missing.	Must be counted and reviewed; not a measured stability finding.
Future columns absent	Not applicable	Not applicable	Current helper may use legacy momentum thresholds and logs a warning.	Test pipeline reachability and do not equate definitions without approval.

Worked technical example: interpret a model comparison row

The following interpretation guide uses repository evaluation behavior and symbolic output names, not fabricated scores. A comparison row contains model identity, accuracy, weighted F1, precision, recall, macro `ROC-AUC` and status. The exact columns and serialization should be confirmed against the run's output. A reader first checks that the candidate and baseline rows refer to the same test set, class encoding, feature schema and source run before comparing values.

Accuracy is calculated from all test predictions. Weighted F1, precision and recall use weighted averaging with `zero_division` set to zero in the implementation. Weighted averaging gives more influence to classes with more true support. Therefore a strong aggregate can coexist with poor recall for a rare class. Always inspect confusion matrices and support counts; do not infer Declining performance from a weighted summary alone.

The evaluation helper initializes macro `ROC-AUC` as unavailable and attempts a multiclass one-versus-rest calculation only when the held-out labels include all encoded classes. If a test partition omits one class, the helper writes a warning that `ROC-AUC` is not statistically meaningful because the split lacks all classes. This is an intentional guard. The comparison should preserve the unavailable state and the warning rather than replacing it with a numeric sentinel that a dashboard might render as a valid score.

The class encoding is 0=Growing, 1=Stable, 2=Declining. A confusion matrix should be displayed in that declared label order, even if a class has zero support, so that readers can identify the absent row or column. Verify labels from `src/common.py` and the model-training

encoder together. A class-order mismatch can make a technically correct matrix semantically wrong.

The evaluation selection rule should be read from the actual code path and comparison artifact. The `README` describes model competition and baselines; the handover verifies which models completed, which were skipped or failed, and how the selected model was chosen. Do not assume a model name in `best_model.joblib` proves the most appropriate model was selected. Retain warnings, candidate failures, tie handling and status fields with the release record.

A reviewer follows one row from the table to its predictions, truth labels and split metadata. The row-level audit checks that each test prediction has a corresponding target, that no training row leaked into test through duplicated time/entity keys, and that the target encoding is reversed correctly for human-readable output. If raw labels or predictions are restricted, retain aggregate evidence and a controlled audit route rather than exporting row-level data.

When test support is low or a class is absent, the immediate recommendation is to restrict interpretation and consult the supervisor. Options may include collecting more approved data, changing the time horizon for a defensible reason, redefining the evaluation estimand or reporting that a metric is unavailable. The choice is methodological and must not be driven only by which alternative produces the highest candidate score.

The final comparison note uses restrained language: it names the run and data mode, states which classes were represented, reports eligible metrics as actually computed, identifies baselines and limitations, and states whether the candidate is exploratory or accepted for a specific use. It does not claim causation, calibration, investment utility or broad generalization unless separate evidence supports those claims.

Illustrative lineage trace from source signal to dashboard

A reviewer can follow one synthetic technology key T across the pipeline to check semantic consistency. In a source fixture, T appears with an approved alias and dated community activity. Ingestion loads the row and records source provenance. Normalization maps the alias to a canonical technology key. Feature engineering aggregates only observations in the declared recent window and stores an observation-month key. The lineage note records the transformation at each step without asserting a real technology or event count.

The label stage separately summarizes activity in the future window under the approved target rule. The future summary is used to construct the training label and must not be copied into the predictor columns. The model receives the approved feature vector and encoded label for training. The evaluation stage holds out the declared later observations and retains the period boundary. This split design is reviewable only if all transformations that learn from data are fit without access to held-out information as required by the protocol.

The prediction artifact stores a technology identifier, predicted class and any probability fields generated by the model. The explanation artifact should retain the same technology and observation identity or a deterministic key to join to it. The dashboard uses the selected model and current artifacts to render the result. The reviewer checks that the key did not change between normalization, prediction, SHAP and display and that no join on row position accidentally associates an explanation with another technology.

For an actual run, the source manifest and artifact timestamps must match the same execution. If the dashboard selects a model based on one file and predictions come from another run, the lineage is broken even if every file parses. The operator should inspect the relevant code paths and naming convention, not assume timestamp proximity proves identity. A proposed manifest can add a shared run identifier if current outputs do not provide sufficient linkage.

This synthetic trace is a diagnostic teaching example. It cannot verify that a production or research source contains the described key or that a future window is complete. For empirical lineage, use an approved record under controlled access, capture only necessary fields and replace direct identifiers with a permitted pseudonymous key. Review whether even a pseudonymous technology-period record could expose restricted information before including it in a handover packet.

The end-to-end lineage decision records each transformation's owner, code location, input/output grain, data mode, rule version and check. A reviewer can then locate a defect: source omissions at ingestion; alias collisions in normalization; duplicate inflation during aggregation; target leakage in feature projection; class remapping in training; invalid metric eligibility in evaluation; key mismatch in explanations; or stale file selection in the dashboard. This map is more useful than a generic statement that all stages were executed.

Table 4.30: Illustrative lineage trace and stage ownership

Stage	Input / transformation	Key invariant	Acceptance evidence
Source / FR-01	Synthetic source row with authorized provenance; loader validates the frame.	Input key and date can be traced to a declared source mode.	Manifest and quality report for the same run.
Normalization / FR-03	Raw alias mapped to canonical technology identifier.	Aliases are intentional and no unrelated technologies collapse.	Mapping fixture and collision review.
Features / FR-03	Observation-window values aggregated to technology-period row.	Grain and cutoff are explicit; only eligible predictors remain.	Feature schema, key count and leakage check.
Target / FR-04	Later-window summary transformed into approved class.	Future target fields never enter estimator predictors.	Threshold version, boundary test and label summary.
Train/evaluate / FR-05-FR-10	Encoded labels and chronological split used for candidate comparison.	Class mapping, split boundary and metric eligibility are preserved.	Split metadata, support counts, warnings and comparison.
Explain/display / FR-11 + dashboard	Selected model and same row key flow to explanation and view.	Model, prediction, feature vector and explanation belong to one run.	Join assertion, artifact identity and user walkthrough.

Final acceptance interview: questions the reviewer should be able to answer

Before accepting the implementation plan for a stated purpose, a reviewer should be able to answer where the generator source lives, how the PDF is rebuilt, what exact nine section titles it contains and where the actual pipeline run is documented. They should be able to distinguish the generator's proposed schedule from an observed delivery schedule and identify which owners approve research meaning, user workflow and release readiness.

For the data path, the reviewer should be able to name the local CSV inputs, identify the six PostgreSQL tables expected by the loader, locate the source manifest and explain what proxy status means. They should know how the project behaves when DATABASE_URL is absent and what failure occurs when a reachable database lacks a required table. They should not need to infer provenance from the model output or dashboard color.

For target and model behavior, the reviewer should be able to explain the recent/future ratio rule, the threshold boundaries, the minimum denominator floor, missing-window handling, legacy fallback and class encoding. They should identify the predictor leakage exclusions, the chronological split boundary and why a missing class makes ROC-AUC unavailable. If they cannot answer, the handover should link the relevant code, tests or methodology record and keep the acceptance condition open.

For operations, the reviewer should be able to run the stated command in the intended environment, find generated artifacts, start the dashboard, interpret the source notice, diagnose a partial run, restore a coherent prior bundle and identify who can approve a research-semantic change. They should know which checks were not performed, including any absent database test, concurrent-user evaluation, user study, security deployment review or production backup service.

For claims, the reviewer should be able to state which outputs support software integration, which support descriptive analysis, which support a held-out model estimate and which remain illustrative. They should know that a development smoke run is not proof of empirical validity and that a successful explanation is not a causal finding. Any handover question without evidence is recorded as unresolved, assigned to an owner and reflected in the scope of use.

Access, retention and evidence minimization

Access should follow the smallest role needed to perform the approved task. The student developer may need source and output access to implement the pipeline; the supervisor needs access to evidence sufficient to review the research decision; the operational user needs only the view and explanation required for the walkthrough. These role boundaries are proposed and must be reconciled with institutional policy and the actual data owner's rules. Do not infer permission from possession of a database credential.

Keep secrets outside source control and generated reports. The .env.example file is a template, not a production secret store. Before sharing logs or screenshots, search for URLs, tokens,

usernames, local file paths and personal data. If a credential is exposed, stop using it, notify the authorized owner and rotate it through the approved process. Preserve a redacted evidence copy for review; do not distribute the unredacted trace merely to make troubleshooting convenient.

Retention applies to source extracts, feature matrices, model bundles, predictions, explanation rows, logs, screenshots and training records. The source owner or institutional policy defines retention; this chapter does not invent a number of days. Record the retention rule, storage location, access owner, backup treatment and disposal method. Derived data can remain sensitive even when direct identifiers are removed, particularly for rare technologies or small time slices.

A proposed evidence packet uses aggregate summaries and synthetic examples where they are sufficient. If a reviewer needs row-level evidence, provide it through an approved restricted location and reference the access-controlled record rather than embedding it in the chapter. Keep samples to the minimum necessary for the decision, document any pseudonymization and assess re-identification risk with the data owner.

Training and change records should not include unnecessary personal details about participants or reporters. Record role, task result and follow-up rather than private opinions unrelated to the project. Restrict access to notes when a user study or institutional policy requires it, and report aggregate observations in public-facing documents only when approved. These steps support reviewability while limiting unnecessary exposure.

Table 4.31: Proposed access and retention decision fields

Data/artifact class	Minimum intended access	Retention authority and location	Disposition evidence
Raw source extract	Approved developer/data steward only; source owner conditions apply.	Data owner or institutional schedule; controlled source location.	Access list, snapshot ID, retention decision and disposal/return record.
Derived feature matrix	Developer and approved research reviewers as needed.	Purpose-limited project storage; preserve only approved fields.	Run identity, lineage, access control and deletion/archival action.
Model and evaluation bundle	Developer, supervisor and intended reviewer.	Project evidence store for period required to reproduce accepted result.	Coherent bundle manifest, version identity and restore decision.
Prediction and SHAP output	Only roles authorized to interpret the result.	Avoid row-level export unless needed and approved.	Join-key minimization, proxy notice and recipient list if shared.
Logs and screenshots	Maintainer and reviewer; redact secrets and local identifiers.	Retain operational evidence under approved project policy.	Redaction check and link to run/change record.
Training/change records	Project roles with a review purpose.	Store task outcomes and decisions; avoid unrelated personal details.	Role-based access and approved closure/disposal evidence.

Monitoring cadence and threshold ownership

Monitoring begins with an owner and a decision the signal can trigger. The repository writes logs and stage outputs, but no automated production monitor or backup service is evidenced by this plan. A proposed review after each run can inspect exit status, manifest, stage completion, missing outputs, class support and warnings. A pre-release review can inspect dependencies, source contract, access and restore readiness. The supervisor selects the cadence that matches the actual use and available resources.

Do not set a numeric alert threshold by intuition when the repository does not define one. For a proposed data-quality alert, review the expected range from an approved source baseline and the consequence of deviation. For class-support monitoring, define the minimum evidence needed to compute each intended metric. For date-gap alerts, define the expected extraction period and treatment of incomplete months. The data owner and supervisor approve these rules before automation or operational reliance.

A monitoring record should distinguish signal, threshold version, observed state, investigation, decision and follow-up. If no alert fired because no monitor exists, say that the check was manual. If an output changed but remained within a threshold, retain enough context to explain the change. A dashboard color or unexplained warning should not become an alert policy without a documented owner and response action.

Review monitor usefulness periodically. A threshold that fires constantly may be ignored; a threshold that never fires may be untested or ineffective. Exercise the alert path using a controlled synthetic condition, confirm delivery to the owner and document how false alarms are cleared. This is a proposed future control, not a claim that TechPulse currently supports alert delivery.

When a monitored signal indicates a material issue, pause promotion of the affected run and open a change or incident record. The response owner assesses source, feature, target, evaluation and dashboard impact. A healthy process exit does not override a data-quality alert. Conversely, a warning with no effect on the approved task can be accepted with a recorded rationale, not silently discarded.

Table 4.32: Proposed manual monitoring and response ownership

Review point	Signal to inspect	Decision authority	Evidence and response
After each run	Exit status, completed stages, manifest mode, required artifacts, warnings and run identity.	Student developer; supervisor for research claim.	Run journal; classify complete/partial; hold incoherent bundle.
Before source refresh	Schema, row/date coverage, missingness, key behavior and source authorization.	Data owner and supervisor.	Snapshot comparison; reconcile or block promotion.
Before model promotion	Class support, eligible metrics, baselines, explanation alignment and wording.	Supervisor.	Evaluation decision record; accept, qualify or defer.
Before interface release	Dashboard artifact linkage, source notice, empty state and user task.	Operational user and release owner.	Walkthrough evidence, release note and rollback trigger.
At restore rehearsal	Bundle completeness, model load, dashboard identity and access route.	Maintainer; supervisor witnesses.	Restore record; retain gap as blocking action if failure.

End-of-project status language

Use status words consistently in the implementation plan and handover. Proposed means a recommendation or baseline to be approved. Observed means directly seen in the checked repository or recorded run. Tested means a named check was executed and its outcome retained. Accepted means the assigned reviewer approved the evidence for a stated use. Provisional means work may continue but a gate remains open. These distinctions prevent future readers from converting a suggested control into a completed activity.

The report's run evidence is an observed development-mode pipeline execution. The periods, effort figures, owners, training, conversion rehearsal, risk scores, test cases, access controls and monitoring cadence are proposals. Illustrative form values and boundary examples are not empirical project results. A later operator should replace proposals with dated records as work is actually performed and preserve the original plan version for traceability.

At submission, the final status summary should identify the exact PDF page count, extracted word count, table and figure totals, generator source path, build command, run mode and verification checks performed. It should also state any unavailable tool or unperformed check. This makes document production itself auditable and avoids implying a check passed when it could not be run.

4.9.5 List of figures and tables

Figure register

Figure 4.1: Proposed seven-period Gantt schedule
Figure 4.2: TechPulse runtime pipeline and dashboard flow
Figure 4.3: Proposed change-management workflow
Figure 4.4: Proposed conversion strategy selection and safeguards

Table register

Table 4.1: Proposed work packages, effort and ownership
Table 4.2: Milestone gates M1-M7 and minimum proceed evidence
Table 4.3: Installation sequence, checks and acceptance evidence
Table 4.4: Software requirements and evidence source
Table 4.5: Conversion activities and release evidence

Table 4.6: Conversion strategy comparison and recommended use
Table 4.7: Rollback triggers and recovery
Table 4.8: Training plan, learning outcomes and effort
Table 4.9: Assessment rubric and pass evidence
Table 4.10: Maintenance categories, triggers and verification
Table 4.11: Change class, verification and communication
Table 4.12: Proposed test cases and observable acceptance criteria
Table 4.13: Illustrative completed change request record
Table 4.14: Illustrative lineage manifest for a rehearsal
Table 4.15: Proposed implementation risks, ordinal score and response
Table 4.16: Proposed A1-H4 task-level work breakdown, dependencies and estimates
Table 4.17: Proposed weekly focus, daily evidence and exit decision
Table 4.18: Proposed artifact acceptance and stale-file control
Table 4.19: Proposed reconciliation fields by required source table
Table 4.20: Illustrative role-based walkthrough record fields
Table 4.21: Proposed release/restore decision gates
Table 4.22: Proposed execution worksheet for representative contract tests
Table 4.23: Proposed feature lineage review template
Table 4.24: Proposed training sessions and role-specific checks
Table 4.25: Proposed quality-warning triage record
Table 4.26: Proposed readiness gates from local development to research use
Table 4.27: Proposed maintenance scenario response and closure evidence
Table 4.28: Proposed implementation review decision record
Table 4.29: Illustrative target-rule boundary examples; no project metrics
Table 4.30: Illustrative lineage trace and stage ownership
Table 4.31: Proposed access and retention decision fields
Table 4.32: Proposed manual monitoring and response ownership

End of Chapter 4